

- [Liu87b] Joseph W.H. Liu. A partial pivoting strategy for sparse symmetric matrix decomposition. *ACM Trans. Math. Software*, 13(2):173–182, 1987.
- [Liu90] Joseph W.H. Liu. The role of elimination trees in sparse factorization. *SIAM J. Matrix Anal. Appl.*, 11(1):134–172, 1990.
- [LS76] Wai-Hung Liu and Andrew H. Sherman. Comparative analysis of the Cuthill–McKee and reverse Cuthill–McKee ordering algorithms for sparse matrices. *SIAM J. Numer. Anal.*, 13(2):198–213, 1976.
- [Mar57] H.M. Markowitz. The elimination form of the inverse and its application to linear programming. *Management Sci.*, 3(3):255–269, 1957.
- [NP93] Esmond G. Ng and Barry W. Peyton. Block sparse Cholesky algorithms on advanced uniprocessor computers. *SIAM J. Sci. Comput.*, 14(5):1034–1056, 1993.
- [NR99] Esmond G. Ng and Padma Raghavan. Performance of greedy ordering heuristics for sparse Cholesky factorization. *SIAM J. Matrix Anal. Appl.*, 20(4):902–914, 1999.
- [PM83] G. Pagallo and C. Maulino. A bipartite quotient graph model for unsymmetric matrices. In V. Pereyra and A. Reinoza, Eds., *Numerical Methods*, vol. 1005, *Lecture Notes in Mathematics*, pp. 227–239. Springer-Verlag, Heidelberg, 1983.
- [PSL90] Alex Pothén, Horst D. Simon, and Kang-Pu Liou. Partitioning sparse matrices with eigenvectors of graphs. *SIAM J. Matrix Anal. Appl.*, 11(3):430–452, 1990.
- [RE98] Edward Rothberg and Stanley C. Eisenstat. Node selection strategies for bottom-up sparse matrix ordering. *SIAM J. Matrix Anal. Appl.*, 19(3):682–695, 1998.
- [RG91] Edward Rothberg and Anoop Gupta. Efficient sparse matrix factorization on high-performance workstations — exploiting the memory hierarchy. *ACM Trans. Math. Software*, 17:313–334, 1991.
- [Ros72] D.J. Rose. A graph-theoretic study of the numerical solution of sparse positive definite systems of linear equations. In R.C. Read, Ed., *Graph Theory and Computing*, pp. 183–217. Academic Press, New York, 1972.
- [RT78] Donald J. Rose and Robert Endre Tarjan. Algorithmic aspects of vertex elimination of directed graphs. *SIAM J. Appl. Math.*, 34(1):176–197, 1978.
- [RTL76] Donald J. Rose, R. Endre Tarjan, and George S. Lueker. Algorithmic aspects of vertex elimination on graphs. *SIAM J. Comput.*, 5(2):266–283, 1976.
- [Sch82] Robert Schreiber. A new implementation of sparse Gaussian elimination. *ACM Trans. Math. Software*, 8(3):256–276, 1982.
- [Sch01] Jürgen Schulze. Towards a tighter coupling of bottom-up and top-down sparse matrix ordering methods. *BIT Num. Math.*, 41(4):800–841, 2001.
- [TW67] W.F. Tinney and J.W. Walker. Direct solutions of sparse network equations by optimally ordered triangular factorization. *Proc. IEEE*, 55:1801–1809, 1967.
- [Yan81] Mihalis Yannakakis. Computing the minimum fill-in is NP-complete. *SIAM J. Alg. Disc. Meth.*, 2(1):77–79, 1981.

41

Iterative Solution Methods for Linear Systems

41.1	Krylov Subspaces and Preconditioners	41-2
41.2	Optimal Krylov Space Methods for Hermitian Problems	41-4
41.3	Optimal and Nonoptimal Krylov Space Methods for Non-Hermitian Problems	41-7
41.4	Preconditioners	41-11
41.5	Preconditioned Algorithms	41-12
41.6	Convergence Rates of CG and MINRES	41-14
41.7	Convergence Rate of GMRES	41-15
41.8	Inexact Preconditioners and Finite Precision Arithmetic, Error Estimation and Stopping Criteria, Text and Reference Books	41-16
	References	41-17

Anne Greenbaum
University of Washington

Given an n by n nonsingular matrix A and an n -vector $\mathbf{b}$, the linear system $A\mathbf{x} = \mathbf{b}$ can always be solved for $\mathbf{x}$ by Gaussian elimination. The work required is approximately $2n^3/3$ operations (additions, subtractions, multiplications, and divisions), and, in general, n^2 words of storage are required. This is often acceptable if n is of moderate size, say $n \leq 1000$, but for much larger values of n , say, $n \approx 10^6$, both the work and storage for Gaussian elimination may become prohibitive.

Where do such large linear systems arise? They may occur in many different areas, but one important source is the numerical solution of partial differential equations (PDEs). Solutions to PDEs can be approximated by replacing derivatives by finite difference quotients. For example, to solve the equation

$$\frac{\partial}{\partial x} \left(a(x, y, z) \frac{\partial u}{\partial x} \right) + \frac{\partial}{\partial y} \left(a(x, y, z) \frac{\partial u}{\partial y} \right) + \frac{\partial}{\partial z} \left(a(x, y, z) \frac{\partial u}{\partial z} \right) = f(x, y, z) \quad \text{in } \Omega$$
$$u = g \quad \text{on boundary of } \Omega,$$

on a three-dimensional region Ω , where a , f , and g are given functions with a bounded away from 0, one might first divide the region Ω into small subregions of width h in each direction, and then replace each partial derivative by a centered difference approximation; e.g.,

$$\frac{\partial}{\partial x} \left(a \frac{\partial u}{\partial x} \right) (x, y, z) \approx \frac{1}{h^2} [a(x + h/2, y, z)(u(x + h, y, z) - u(x, y, z)) \\ - a(x - h/2, y, z)(u(x, y, z) - u(x - h, y, z))],$$

with similar approximations for $\partial/\partial y(a\partial u/\partial y)$ and $\partial/\partial z(a\partial u/\partial z)$. If the resulting finite difference approximation to the differential operator is set equal to the right-hand side value $f(x_i, y_j, z_k)$ at each of the interior mesh points (x_i, y_j, z_k) , $i = 1, \dots, n_1$, $j = 1, \dots, n_2$, $k = 1, \dots, n_3$, then this gives a system of $n = n_1 n_2 n_3$ linear equations for the n unknown values of u at these mesh points. If u_{ijk} denotes the approximation to $u(x_i, y_j, z_k)$, then the equations are

$$\begin{aligned} \frac{1}{h^2} [& a(x_i + h/2, y_j, z_k)(u_{i+1,j,k} - u_{ijk}) - a(x_i - h/2, y_j, z_k)(u_{ijk} - u_{i-1,j,k}) \\ & + a(x_i, y_j + h/2, z_k)(u_{i,j+1,k} - u_{ijk}) - a(x_i, y_j - h/2, z_k)(u_{ijk} - u_{i,j-1,k}) \\ & + a(x_i, y_j, z_k + h/2)(u_{i,j,k+1} - u_{ijk}) - a(x_i, y_j, z_k - h/2)(u_{ijk} - u_{i,j,k-1})] \\ & = f(x_i, y_j, z_k). \end{aligned}$$

The formula must be modified near the boundary of the region, where known boundary values are added to the right-hand side. Still the result is a system of linear equations for the unknown interior values of u . If $n_1 = n_2 = n_3 = 100$, then the number of equations and unknowns is $100^3 = 10^6$.

Notice, however, that the system of linear equations is *sparse*; each equation involves only a few (in this case seven) of the unknowns. The actual form of the system matrix A depends on the numbering of equations and unknowns. Using the *natural ordering*, equations and unknowns are ordered first by i , then j , then k . The result is a *banded* matrix, whose bandwidth is approximately $n_1 n_2$, since unknowns in any z plane couple only to those in the same and adjacent z planes. This results in some savings for Gaussian elimination. Only entries inside the band need be stored because these are the only ones that *fill in* (become nonzero, even if originally they were zero) during the process. The resulting work is about $2(n_1 n_2)^2 n$ operations, and the storage required is about $n_1 n_2 n$ words. Still, this is too much when $n_1 = n_2 = n_3 = 100$. Different orderings can be used to further reduce fill in, but another option is to use iterative methods.

Because the matrix is so sparse, matrix-vector multiplication is very cheap. In the above example, the product of the matrix with a given vector can be accomplished with just $7n$ multiplications and $6n$ additions. The nonzeros of the matrix occupy only $7n$ words and, in this case, they are so simple that they hardly need be stored at all. If the linear system $Ax = b$ could be solved iteratively, using only matrix-vector multiplication and, perhaps, solution of some much simpler linear systems such as diagonal or sparse triangular systems, then a tremendous savings might be achieved in both work and storage.

This section describes how to solve such systems iteratively. While iterative methods are appropriate for sparse systems like the one above, they also may be useful for *structured systems*. If matrix-vector multiplication can be performed rapidly, and if the structure of the matrix is such that it is not necessary to store the entire matrix but only certain parts or values in order to carry out the matrix-vector multiplication, then iterative methods may be faster and require less storage than Gaussian elimination or other methods for solving $Ax = b$.

41.1 Krylov Subspaces and Preconditioners

Definitions:

An **iterative method** for solving a linear system $Ax = b$ is an algorithm that starts with an initial guess x_0 for the solution and successively modifies that guess in an attempt to obtain improved approximate solutions $x_1, x_2, \dots$.

The **residual** at step k of an iterative method for solving $Ax = b$ is the vector $r_k \equiv b - Ax_k$, where x_k is the approximate solution generated at step k . The initial residual is $r_0 \equiv b - Ax_0$, where x_0 is the initial guess for the solution.

The **error** at step k is the difference between the true solution $A^{-1}b$ and the approximate solution x_k : $e_k \equiv A^{-1}b - x_k$.

A **Krylov space** is a space of the form $\text{span}\{q, Aq, A^2q, \dots, A^{k-1}q\}$, where A is an n by n matrix and q is an n -vector. This space will be denoted as $K_k(A, q)$.

A **preconditioner** is a matrix M designed to improve the performance of an iterative method for solving the linear system $A\mathbf{x} = \mathbf{b}$. Linear systems with coefficient matrix M should be easier to solve than the original linear system, since such systems will be solved at each iteration.

The matrix $M^{-1}A$ (for **left preconditioning**) or AM^{-1} (for **right preconditioning**) or $L^{-1}AL^{-*}$ (for **Hermitian preconditioning**, when $M = LL^*$) is sometimes referred to as the **preconditioned iteration matrix**.

Another name for a preconditioner is a **splitting**; that is, if A is written in the form $A = M - N$, then this is referred to as a splitting of A , and iterative methods based on this splitting are equivalent to methods using M as a preconditioner.

A **regular splitting** is one for which M is nonsingular with $M^{-1} \geq 0$ (elementwise) and $M \geq A$ (elementwise).

Facts:

The following facts and general information on Krylov spaces and preconditioners can be found, for example, in [Axe95], [Gre97], [Hac94], [Saa03], and [Vor03].

1. An iterative method may obtain the exact solution at some stage (in which case it might be considered a direct method), but it may still be thought of as an iterative method because the user is interested in obtaining a good approximate solution before the exact solution is reached.
2. Each iteration of an iterative method usually requires one or more matrix-vector multiplications, using the matrix A and possibly its Hermitian transpose A^* . An iteration may also require the solution of a preconditioning system $M\mathbf{z} = \mathbf{r}$.
3. The residual and error vector at step k of an iterative method are related by $\mathbf{r}_k = A\mathbf{e}_k$.
4. All of the iterative methods to be described in this chapter generate approximate solutions $\mathbf{x}_k$, $k = 1, 2, \dots$, such that $\mathbf{x}_k - \mathbf{x}_0$ lies in the Krylov space $\text{span}\{\mathbf{z}_0, C\mathbf{z}_0, \dots, C^{k-1}\mathbf{z}_0\}$, where $\mathbf{z}_0$ is the initial residual, possibly multiplied by a preconditioner, and C is the preconditioned iteration matrix.
5. The *Jacobi*, *Gauss-Seidel*, and *SOR (successive overrelaxation)* methods use the simple iteration

$$\mathbf{x}_k = \mathbf{x}_{k-1} + M^{-1}(\mathbf{b} - A\mathbf{x}_{k-1}), \quad k = 1, 2, \dots,$$

with different preconditioners M . For the Jacobi method, M is taken to be the diagonal of A , while for the Gauss-Seidel method, M is the lower triangle of A . For the SOR method, M is of the form $\omega^{-1}D - L$, where D is the diagonal of A , $-L$ is the strict lower triangle of A , and ω is a relaxation parameter. Subtracting each side of this equation from the true solution $A^{-1}\mathbf{b}$, we find that the error at step k is

$$\mathbf{e}_k = (I - M^{-1}A)\mathbf{e}_{k-1} = \dots = (I - M^{-1}A)^k \mathbf{e}_0.$$

Subtracting each side of this equation from $\mathbf{e}_0$, we find that $\mathbf{x}_k$ satisfies

$$\begin{aligned} \mathbf{e}_0 - \mathbf{e}_k &= \mathbf{x}_k - \mathbf{x}_0 = [I - (I - M^{-1}A)^k] \mathbf{e}_0 \\ &= \left[\sum_{j=1}^k \binom{k}{j} (-1)^{j-1} (M^{-1}A)^{j-1} \right] \mathbf{z}_0, \end{aligned}$$

where $\mathbf{z}_0 = M^{-1}A\mathbf{e}_0 = M^{-1}\mathbf{r}_0$. Thus, $\mathbf{x}_k - \mathbf{x}_0$ lies in the Krylov space

$$\text{span}\{\mathbf{z}_0, (M^{-1}A)\mathbf{z}_0, \dots, (M^{-1}A)^{k-1}\mathbf{z}_0\}.$$

6. Standard *multigrid methods* for solving linear systems arising from partial differential equations are also of the form $\mathbf{x}_k = \mathbf{x}_{k-1} + M^{-1}\mathbf{r}_{k-1}$. For these methods, computing $M^{-1}\mathbf{r}_{k-1}$ involves restricting the residual to a coarser grid or grids, solving (or iterating) with the linear system on those grids, and then prolonging the solution back to the finest grid.

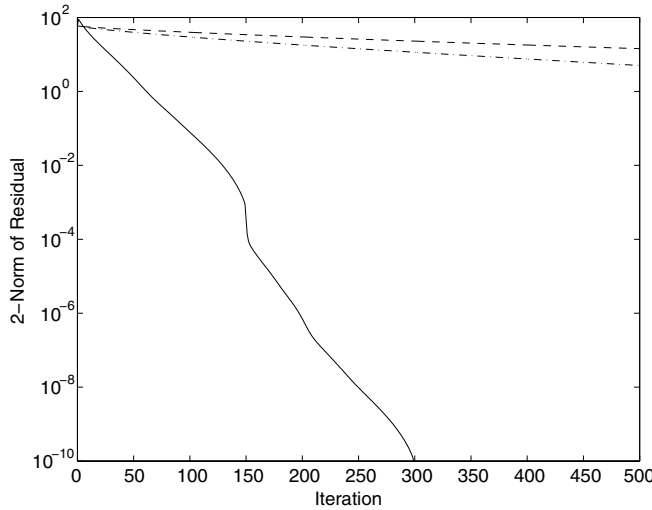


FIGURE 41.1 Convergence of iterative methods for the problem given in the introduction with $a(x, y, z) = 1 + x + 3yz$, $h = 1/50$. Jacobi (dashed), Gauss–Seidel (dash-dot), and SOR with $\omega = 1.9$ (solid).

Applications:

1. Figure 41.1 shows the convergence of the Jacobi, Gauss–Seidel, and SOR (with $\omega = 1.9$) iterative methods for the problem described at the beginning of this chapter, using a mildly varying coefficient $a(x, y, z) = 1 + x + 3yz$ on the unit cube $\Omega = [0, 1] \times [0, 1] \times [0, 1]$ with homogeneous Dirichlet boundary conditions, $u = 0$ on $\partial\Omega$. The right-hand side function f was chosen so that the solution to the differential equation would be $u(x, y, z) = x(1 - x)y^2(1 - y)z(1 - z)^2$. The region was discretized using a $50 \times 50 \times 50$ mesh, and the natural ordering of nodes was used, along with a zero initial guess.

41.2 Optimal Krylov Space Methods for Hermitian Problems

Throughout this section, we let A and b denote the already preconditioned matrix and right-hand side vector, and we assume that A is Hermitian. Note that if the original coefficient matrix is Hermitian, then this requires Hermitian positive definite preconditioning (preconditioner of the form $M = LL^*$ and preconditioned matrix of the form $L^{-1}AL^{-*}$) in order to maintain this property.

Definitions:

The **Minimal Residual (MINRES)** algorithm generates, at each step k , the approximation $\mathbf{x}_k$ with $\mathbf{x}_k - \mathbf{x}_0 \in K_k(A, \mathbf{r}_0)$ for which the 2-norm of the residual, $\|\mathbf{r}_k\| \equiv \langle \mathbf{r}_k, \mathbf{r}_k \rangle^{1/2}$, is minimal.

The **Conjugate Gradient (CG)** algorithm for Hermitian positive definite matrices generates, at each step k , the approximation $\mathbf{x}_k$ with $\mathbf{x}_k - \mathbf{x}_0 \in K_k(A, \mathbf{r}_0)$ for which the A -norm of the error, $\|\mathbf{e}_k\|_A \equiv \langle \mathbf{e}_k, A\mathbf{e}_k \rangle^{1/2}$, is minimal. (Note that this is sometimes referred to as the $A^{1/2}$ -norm of the error, e.g., in Chapter 37 of this book.)

The **Lanczos** algorithm for Hermitian matrices is a short recurrence for constructing an orthonormal basis for a Krylov space.

Facts:

The following facts can be found in any of the general references [Axe95], [Gre97], [Hac94], [Saa03], and [Vor03].

1. The Lanczos algorithm [Lan50] is implemented as follows:

Lanczos Algorithm. (For Hermitian matrices A)

Given $\mathbf{q}_1$ with $\|\mathbf{q}_1\| = 1$, set $\beta_0 = 0$. For $j = 1, 2, \dots$,

$$\tilde{\mathbf{q}}_{j+1} = A\mathbf{q}_j - \beta_{j-1}\mathbf{q}_{j-1}. \text{ Set } \alpha_j = \langle \tilde{\mathbf{q}}_{j+1}, \mathbf{q}_j \rangle, \quad \tilde{\mathbf{q}}_{j+1} \leftarrow \tilde{\mathbf{q}}_{j+1} - \alpha_j \mathbf{q}_j.$$

$$\beta_j = \|\tilde{\mathbf{q}}_{j+1}\|, \quad \mathbf{q}_{j+1} = \tilde{\mathbf{q}}_{j+1}/\beta_j.$$

2. It can be shown by induction that the Lanczos vectors $\mathbf{q}_1, \mathbf{q}_2, \dots$ produced by the above algorithm are orthogonal. Gathering the first k vectors together as the columns of an n by k matrix Q_k , this recurrence can be written succinctly in the form

$$AQ_k = Q_k T_k + \beta_k \mathbf{q}_{k+1} \xi_k^T,$$

where $\xi_k \equiv (0, \dots, 0, 1)^T$ is the k th unit vector and T_k is the tridiagonal matrix of recurrence coefficients:

$$T_k \equiv \begin{pmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \ddots & \ddots & \\ & \ddots & \ddots & \beta_{k-1} \\ & & \beta_{k-1} & \alpha_k \end{pmatrix}.$$

The above equation is sometimes written in the form

$$AQ_k = Q_{k+1} \underline{T}_k,$$

where $\underline{T}_k$ is the $k+1$ by k matrix whose top k by k block is T_k and whose bottom row is zero except for the last entry which is β_k .

3. If the initial vector $\mathbf{q}_1$ in the Lanczos algorithm is taken to be $\mathbf{q}_1 = \mathbf{r}_0/\|\mathbf{r}_0\|$, then the columns of Q_k span the Krylov space $K_k(A, \mathbf{r}_0)$. Both the MINRES and CG algorithms take the approximation $\mathbf{x}_k$ to be of the form $\mathbf{x}_0 + Q_k \mathbf{y}_k$ for a certain vector $\mathbf{y}_k$. For the MINRES algorithm, $\mathbf{y}_k$ is the solution of the $k+1$ by k least squares problem

$$\min_{\mathbf{y}} \|\beta \xi_1 - \underline{T}_k \mathbf{y}\|,$$

where $\beta \equiv \|\mathbf{r}_0\|$ and $\xi_1 \equiv (1, 0, \dots, 0)^T$ is the first unit vector. For the CG algorithm, $\mathbf{y}_k$ is the solution of the k by k tridiagonal system

$$T_k \mathbf{y} = \beta \xi_1.$$

4. The following algorithms are standard implementations of the CG and MINRES methods.

Conjugate Gradient Method (CG).

(For Hermitian Positive Definite Problems)

Given an initial guess $\mathbf{x}_0$, compute $\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$ and set $\mathbf{p}_0 = \mathbf{r}_0$. For $k = 1, 2, \dots$,

Compute $A\mathbf{p}_{k-1}$.

Set $\mathbf{x}_k = \mathbf{x}_{k-1} + a_{k-1}\mathbf{p}_{k-1}$, where $a_{k-1} = \frac{\langle \mathbf{r}_{k-1}, \mathbf{r}_{k-1} \rangle}{\langle \mathbf{p}_{k-1}, A\mathbf{p}_{k-1} \rangle}$.

Compute $\mathbf{r}_k = \mathbf{r}_{k-1} - a_{k-1}A\mathbf{p}_{k-1}$.

Set $\mathbf{p}_k = \mathbf{r}_k + b_{k-1}\mathbf{p}_{k-1}$, where $b_{k-1} = \frac{\langle \mathbf{r}_k, \mathbf{r}_k \rangle}{\langle \mathbf{r}_{k-1}, \mathbf{r}_{k-1} \rangle}$.

Minimal Residual Algorithm (MINRES). (For Hermitian Problems)

Given $\mathbf{x}_0$, compute $\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$ and set $\mathbf{q}_1 = \mathbf{r}_0 / \|\mathbf{r}_0\|$.

Initialize $\xi = (1, 0, \dots, 0)^T$, $\beta = \|\mathbf{r}_0\|$. For $k = 1, 2, \dots$,

Compute $\mathbf{q}_{k+1}$, $\alpha_k \equiv T(k, k)$, and $\beta_k \equiv T(k+1, k) \equiv T(k, k+1)$ using the Lanczos algorithm.

Apply rotations F_{k-2} and F_{k-1} to the last column of T ; that is,

$$\begin{pmatrix} T(k-2, k) \\ T(k-1, k) \end{pmatrix} \leftarrow \begin{pmatrix} c_{k-2} & s_{k-2} \\ -\bar{s}_{k-2} & c_{k-2} \end{pmatrix} \begin{pmatrix} 0 \\ T(k-1, k) \end{pmatrix}, \text{ if } k > 2,$$

$$\begin{pmatrix} T(k-1, k) \\ T(k, k) \end{pmatrix} \leftarrow \begin{pmatrix} c_{k-1} & s_{k-1} \\ -\bar{s}_{k-1} & c_{k-1} \end{pmatrix} \begin{pmatrix} T(k-1, k) \\ T(k, k) \end{pmatrix}, \text{ if } k > 1.$$

Compute the k^{th} rotation, c_k and s_k , to annihilate the $(k+1, k)$ entry of T :

$$c_k = |T(k, k)| / \sqrt{|T(k, k)|^2 + |T(k+1, k)|^2}, \bar{s}_k = c_k T(k+1, k) / T(k, k).$$

Apply k^{th} rotation to ξ and to last column of T :

$$\begin{pmatrix} \xi(k) \\ \xi(k+1) \end{pmatrix} \leftarrow \begin{pmatrix} c_k & s_k \\ -\bar{s}_k & c_k \end{pmatrix} \begin{pmatrix} \xi(k) \\ 0 \end{pmatrix}.$$

$$T(k, k) \leftarrow c_k T(k, k) + s_k T(k+1, k), \quad T(k+1, k) \leftarrow 0.$$

Compute $\mathbf{p}_{k-1} = [\mathbf{q}_k - T(k-1, k)\mathbf{p}_{k-2} - T(k-2, k)\mathbf{p}_{k-3}] / T(k, k)$, where undefined terms are zero for $k \leq 2$.

Set $\mathbf{x}_k = \mathbf{x}_{k-1} + a_{k-1}\mathbf{p}_{k-1}$, where $a_{k-1} = \beta\xi(k)$.

5. In exact arithmetic, both the CG and the MINRES algorithms obtain the exact solution in at most n steps, since the affine space $\mathbf{x}_0 + K_n(A, \mathbf{r}_0)$ contains the true solution.

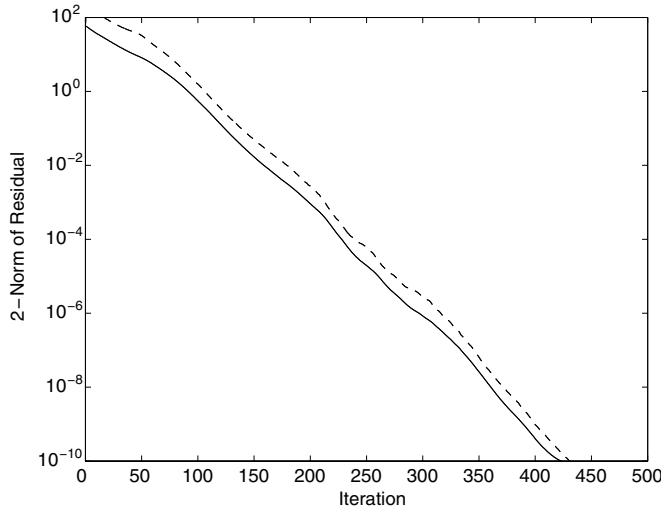


FIGURE 41.2 Convergence of MINRES (solid) and CG (dashed) for the problem given in the introduction with $a(x, y, z) = 1 + x + 3yz$, $h = 1/50$.

Applications:

- Figure 41.2 shows the convergence (in terms of the 2-norm of the residual) of the (unpreconditioned) CG and MINRES algorithms for the same problem used in the previous section.

Note that the 2-norm of the residual decreases monotonically in the MINRES algorithm, but not in the CG algorithm. Had we instead plotted the A -norm of the error, then the CG convergence curve would have been below that for MINRES.

41.3 Optimal and Nonoptimal Krylov Space Methods for Non-Hermitian Problems

In this section, we again let A and $\mathbf{b}$ denote the already preconditioned matrix and right-hand side vector. The matrix A is assumed to be a general nonsingular n by n matrix.

Definitions:

The **Generalized Minimal Residual (GMRES)** algorithm generates, at each step k , the approximation $\mathbf{x}_k$ with $\mathbf{x}_k - \mathbf{x}_0 \in K_k(A, \mathbf{r}_0)$ for which the 2-norm of the residual is minimal.

The **Full Orthogonalization Method (FOM)** generates, at each step k , the approximation $\mathbf{x}_k$ with $\mathbf{x}_k - \mathbf{x}_0 \in K_k(A, \mathbf{r}_0)$ for which the residual is orthogonal to the Krylov space $K_k(A, \mathbf{r}_0)$.

The **Arnoldi algorithm** is a method for constructing an orthonormal basis for a Krylov space that requires saving all of the basis vectors and orthogonalizing against them at each step.

The **restarted GMRES algorithm**, $\text{GMRES}(j)$, is defined by simply restarting GMRES every j steps, using the latest iterate as the initial guess for the next GMRES cycle. Sometimes partial information from the previous GMRES cycle is retained and used after the restart.

The **non-Hermitian (or two-sided) Lanczos algorithm** uses a pair of three-term recurrences involving A and A^* to construct **biorthogonal** bases for the Krylov spaces $K_k(A, \mathbf{r}_0)$ and $K_k(A^*, \hat{\mathbf{r}}_0)$, where $\hat{\mathbf{r}}_0$ is a given vector with $\langle \mathbf{r}_0, \hat{\mathbf{r}}_0 \rangle \neq 0$. If the vectors $\mathbf{v}_1, \dots, \mathbf{v}_k$ are the basis vectors for $K_k(A, \mathbf{r}_0)$, and $\mathbf{w}_1, \dots, \mathbf{w}_k$ are the basis vectors for $K_k(A^*, \hat{\mathbf{r}}_0)$, then $\langle \mathbf{v}_i, \mathbf{w}_j \rangle = 0$ for $i \neq j$.

In the **BiCG** (biconjugate gradient) method, the approximate solution $\mathbf{x}_k$ is chosen so that the residual $\mathbf{r}_k$ is orthogonal to $K_k(A^*, \hat{\mathbf{r}}_0)$.

In the **QMR** (quasi-minimal residual) algorithm, the approximate solution $\mathbf{x}_k$ is chosen to minimize a quantity that is related to (but not necessarily equal to) the residual norm.

The **CGS** (conjugate gradient squared) algorithm constructs an approximate solution $\mathbf{x}_k$ for which $\mathbf{r}_k = \varphi_k^2(A)\mathbf{r}_0$, where $\varphi_k(A)$ is the k th degree polynomial constructed in the BiCG algorithm; that is, the BiCG residual at step k is $\varphi_k(A)\mathbf{r}_0$.

The **BiCGSTAB** algorithm combines CGS with a one or more step residual norm minimizing method to smooth out the convergence.

Facts:

1. The Arnoldi algorithm [Arn51] is implemented as follows:

Arnoldi Algorithm.

Given $\mathbf{q}_1$ with $\|\mathbf{q}_1\| = 1$. For $j = 1, 2, \dots$,

$$\tilde{\mathbf{q}}_{j+1} = A\mathbf{q}_j. \text{ For } i = 1, \dots, j, \quad h_{ij} = \langle \tilde{\mathbf{q}}_{j+1}, \mathbf{q}_i \rangle, \quad \tilde{\mathbf{q}}_{j+1} \leftarrow \tilde{\mathbf{q}}_{j+1} - h_{ij}\mathbf{q}_i.$$

$$h_{j+1,j} = \|\tilde{\mathbf{q}}_{j+1}\|, \quad \mathbf{q}_{j+1} = \tilde{\mathbf{q}}_{j+1}/h_{j+1,j}.$$

2. Unlike the Hermitian case, if A is non-Hermitian then there is no known algorithm for finding the optimal approximations from successive Krylov spaces, while performing only $O(n)$ operations per iteration. In fact, a theorem due to Faber and Manteuffel [FM84] shows that for most non-Hermitian matrices A there is no short recurrence that generates these optimal approximations for successive values $k = 1, 2, \dots$. Hence, the current options for non-Hermitian problems are either to perform extra work ($O(nk)$ operations at step k) and use extra storage ($O(nk)$ words to perform k iterations) to find optimal approximations from the successive Krylov subspaces or to settle for nonoptimal approximations. The (full) GMRES (generalized minimal residual) algorithm [SS86] finds the approximation for which the 2-norm of the residual is minimal, at the cost of this extra work and storage, while other non-Hermitian iterative methods (e.g., BiCG [Fle75], CGS [Son89], QMR [FN91], BiCGSTAB [Vor92], and restarted GMRES [SS86], [Mor95], [DeS99]) generate nonoptimal approximations.
3. Similar to the MINRES algorithm, the GMRES algorithm uses the Arnoldi iteration defined above to construct an orthonormal basis for the Krylov space $K_k(A, \mathbf{r}_0)$.

If Q_k is the n by k matrix with the orthonormal basis vectors $\mathbf{q}_1, \dots, \mathbf{q}_k$ as columns, then the Arnoldi iteration can be written simply as

$$AQ_k = Q_k H_k + h_{k+1,k} \mathbf{q}_{k+1} \xi_k^T = Q_{k+1} \underline{H}_k.$$

Here H_k is the k by k upper Hessenberg matrix with (i, j) entry equal to h_{ij} , and $\underline{H}_k$ is the $k+1$ by k matrix whose upper k by k block is H_k and whose bottom row is zero except for the last entry, which is $h_{k+1,k}$.

If $\mathbf{q}_1 = \mathbf{r}_0/\|\mathbf{r}_0\|$, then the columns of Q_k span the Krylov space $K_k(A, \mathbf{r}_0)$, and the GMRES approximation is taken to be of the form $\mathbf{x}_k = \mathbf{x}_0 + Q_k \mathbf{y}_k$ for some vector $\mathbf{y}_k$. To minimize the 2-norm of the residual, the vector $\mathbf{y}_k$ is chosen to solve the least squares problem

$$\min_{\mathbf{y}} \|\beta \xi_1 - \underline{H}_k \mathbf{y}\|, \quad \beta \equiv \|\mathbf{r}_0\|.$$

The GMRES algorithm [SS86] can be implemented as follows:

Generalized Minimal Residual Algorithm (GMRES).

Given $\mathbf{x}_0$, compute $\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$ and set $\mathbf{q}_1 = \mathbf{r}_0 / \|\mathbf{r}_0\|$.

Initialize $\xi = (1, 0, \dots, 0)^T$, $\beta = \|\mathbf{r}_0\|$. For $k = 1, 2, \dots$,

Compute $\mathbf{q}_{k+1}$ and $h_{i,k} \equiv H(i, k)$, $i = 1, \dots, k+1$, using the Arnoldi algorithm.

Apply rotations $F_1, \dots, F_{k-1}$ to the last column of H ; that is,

For $i = 1, \dots, k-1$,

$$\begin{pmatrix} H(i, k) \\ H(i+1, k) \end{pmatrix} \leftarrow \begin{pmatrix} c_i & s_i \\ -\bar{s}_i & c_i \end{pmatrix} \begin{pmatrix} H(i, k) \\ H(i+1, k) \end{pmatrix}.$$

Compute the k^{th} rotation, c_k and s_k , to annihilate the $(k+1, k)$ entry of H :

$c_k = |H(k, k)| / \sqrt{|H(k, k)|^2 + |H(k+1, k)|^2}$, $\bar{s}_k = c_k H(k+1, k) / H(k, k)$.

Apply k^{th} rotation to ξ and to last column of H :

$$\begin{pmatrix} \xi(k) \\ \xi(k+1) \end{pmatrix} \leftarrow \begin{pmatrix} c_k & s_k \\ -\bar{s}_k & c_k \end{pmatrix} \begin{pmatrix} \xi(k) \\ 0 \end{pmatrix}$$

$$H(k, k) \leftarrow c_k H(k, k) + s_k H(k+1, k), \quad H(k+1, k) \leftarrow 0.$$

If residual norm estimate $\beta|\xi(k+1)|$ is sufficiently small, then

Solve upper triangular system $H_{k \times k} \mathbf{y}_k = \beta \xi_{k \times 1}$.

Compute $\mathbf{x}_k = \mathbf{x}_0 + Q_k \mathbf{y}_k$.

4. The (full) GMRES algorithm described above may be impractical because of increasing storage and work requirements, if the number of iterations needed to solve the linear system is large. In this case, the restarted GMRES algorithm or one of the algorithms based on the non-Hermitian Lanczos process may provide a reasonable alternative. The BiCGSTAB algorithm [Vor92] is often among the most effective iteration methods for solving non-Hermitian linear systems. The algorithm can be written as follows:

BiCGSTAB.

Given $\mathbf{x}_0$, compute $\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$ and set $\mathbf{p}_0 = \mathbf{r}_0$. Choose $\hat{\mathbf{r}}_0$ such that $\langle \mathbf{r}_0, \hat{\mathbf{r}}_0 \rangle \neq 0$. For $k = 1, 2, \dots$,

Compute $A\mathbf{p}_{k-1}$.

Set $\mathbf{x}_{k-1/2} = \mathbf{x}_{k-1} + a_{k-1}\mathbf{p}_{k-1}$, where $a_{k-1} = \frac{\langle \mathbf{r}_{k-1}, \hat{\mathbf{r}}_0 \rangle}{\langle A\mathbf{p}_{k-1}, \hat{\mathbf{r}}_0 \rangle}$.

Compute $\mathbf{r}_{k-1/2} = \mathbf{r}_{k-1} - a_{k-1}A\mathbf{p}_{k-1}$.

Compute $A\mathbf{r}_{k-1/2}$.

Set $\mathbf{x}_k = \mathbf{x}_{k-1/2} + \omega_k \mathbf{r}_{k-1/2}$, where $\omega_k = \frac{\langle \mathbf{r}_{k-1/2}, A\mathbf{r}_{k-1/2} \rangle}{\langle A\mathbf{r}_{k-1/2}, A\mathbf{r}_{k-1/2} \rangle}$.

Compute $\mathbf{r}_k = \mathbf{r}_{k-1/2} - \omega_k A\mathbf{r}_{k-1/2}$.

Compute $\mathbf{p}_k = \mathbf{r}_k + b_k(\mathbf{p}_{k-1} - \omega_k A\mathbf{p}_{k-1})$, where $b_k = \frac{a_{k-1}}{\omega_k} \frac{\langle \mathbf{r}_k, \hat{\mathbf{r}}_0 \rangle}{\langle \mathbf{r}_{k-1}, \hat{\mathbf{r}}_0 \rangle}$.

5. The non-Hermitian Lanczos algorithm can break down if $\langle \mathbf{v}_i, \mathbf{w}_i \rangle = 0$, but neither $\mathbf{v}_i$ nor $\mathbf{w}_i$ is zero. In this case **look-ahead** strategies have been devised to skip steps at which the Lanczos vectors are undefined. See, for instance, [PTL85], [Nac91], and [FN91]. These look-ahead procedures are used in the QMR algorithm.
6. When A is Hermitian and $\hat{\mathbf{r}}_0 = \mathbf{r}_0$, the BiCG method reduces to the CG algorithm, while the QMR method reduces to the MINRES algorithm.
7. The question of which iterative method to use is, of course, an important one. Unfortunately, there is no straightforward answer. It is problem dependent and may depend also on the type of machine being used. If matrix-vector multiplication is very expensive (e.g., if A is dense and has no special properties to enable fast matrix-vector multiplication), then full GMRES is probably the method of choice because it requires the fewest matrix-vector multiplications to reduce the residual norm to a desired level. If matrix-vector multiplication is not so expensive or if storage becomes a problem for full GMRES, then a restarted GMRES algorithm, some variant of the QMR method, or some variant of BiCGSTAB may be a reasonable alternative. With a sufficiently good preconditioner, each of these iterative methods can be expected to find a good approximate solution quickly. In fact, with a sufficiently good preconditioner M , an even simpler iteration method such as $\mathbf{x}_k = \mathbf{x}_{k-1} + M^{-1}(\mathbf{b} - A\mathbf{x}_{k-1})$ may converge in just a few iterations, and this avoids the cost of inner products and other things in the more sophisticated Krylov space methods.

Applications:

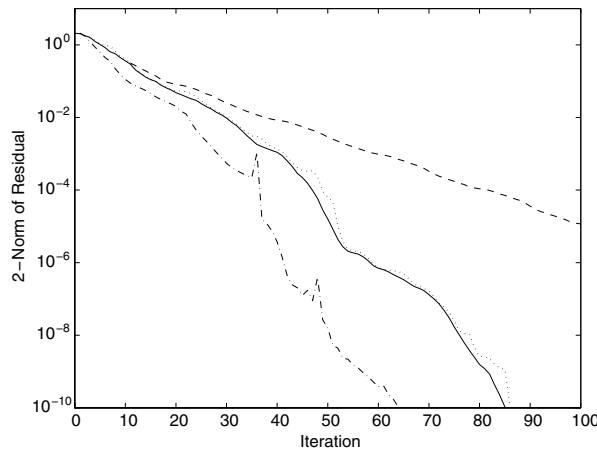


FIGURE 41.3 Convergence of full GMRES (solid), restarted GMRES (restarted every 10 steps) (dashed), QMR (dotted), and BiCGSTAB (dash-dot) for a problem from neutron transport. For GMRES (full or restarted), the number of matrix-vector multiplications is the same as the number of iterations, while for QMR and BiCGSTAB, the number of matrix-vector multiplications is *twice* the number of iterations.

1. To illustrate the behavior of iterative methods for solving non-Hermitian linear systems, we have taken a simple problem involving the Boltzmann transport equation in one dimension:

$$\mu \frac{\partial \psi}{\partial x} + \sigma_T \psi - \sigma_s \phi = f, \quad x \in [a, b], \quad \mu \in [-1, 1],$$

where

$$\phi(x) = \frac{1}{2} \int_{-1}^1 \psi(x, \mu') d\mu',$$

with boundary conditions

$$\begin{aligned} \psi(b, \mu) &= \psi_b(\mu), \quad -1 \leq \mu < 0, \\ \psi(a, \mu) &= \psi_a(\mu), \quad 0 < \mu \leq 1. \end{aligned}$$

The difference method used is described in [Gre97], and a test problem from [ML82] was solved. Figure 41.3 shows the convergence of full GMRES, restarted GMRES (restarted every 10 steps), QMR, and BiCGSTAB. One should keep in mind that each iteration of the QMR algorithm requires *two* matrix-vector multiplications, one with A and one with A^* . Still, the QMR approximation at iteration k lies in the k -dimensional affine space $\mathbf{x}_0 + \text{span}\{\mathbf{r}_0, A\mathbf{r}_0, \dots, A^{k-1}\mathbf{r}_0\}$. Each iteration of the BiCGSTAB algorithm requires *two* matrix-vector multiplications with A , and the approximate solution generated at step k lies in the $2k$ -dimensional affine space $\mathbf{x}_0 + \text{span}\{\mathbf{r}_0, A\mathbf{r}_0, \dots, A^{2k-1}\mathbf{r}_0\}$. The full GMRES algorithm finds the optimal approximation from this space at step $2k$. Thus, the GMRES residual norm at step $2k$ is guaranteed to be less than or equal to the BiCGSTAB residual norm at step k , and each requires the *same* number of matrix-vector multiplications to compute.

41.4 Preconditioners

Definitions:

An **incomplete Cholesky decomposition** is a preconditioner for a Hermitian positive definite matrix A of the form $M = LL^*$, where L is a sparse lower triangular matrix. The entries of L are chosen so that certain entries of LL^* match those of A . If L is taken to have the same sparsity pattern as the lower triangle of A , then its entries are chosen so that LL^* matches A in the positions where A has nonzeros.

A **modified incomplete Cholesky decomposition** is a preconditioner of the same form $M = LL^*$ as the incomplete Cholesky preconditioner, but the entries of L are modified so that instead of having M match as many entries of A as possible, the preconditioner M has certain other properties, such as the same row sums as A .

An **incomplete LU decomposition** is a preconditioner for a general matrix A of the form $M = LU$, where L and U are sparse lower and upper triangular matrices, respectively. The entries of L and U are chosen so that certain entries of LU match the corresponding entries of A .

A **sparse approximate inverse** is a sparse matrix M^{-1} constructed to approximate A^{-1} .

A **multigrid** preconditioner is a preconditioner designed for problems arising from partial differential equations discretized on grids. Solving the preconditioning system $Mz = r$ entails *restricting* the residual to coarser grids, performing relaxation steps for the linear system corresponding to the same differential operator on the coarser grids, and *prolonging* solutions back to finer grids.

An **algebraic multigrid** preconditioner is a preconditioner that uses principles similar to those used for PDE problems on grids, when the “grid” for the problem is unknown or nonexistent and only the matrix is available.

Facts:

1. If A is an M-matrix, then for every subset S of off-diagonal indices there exists a lower triangular matrix $L = [l_{ij}]$ with unit diagonal and an upper triangular matrix $U = [u_{ij}]$ such that $A = LU - R$, where

$$l_{ij} = 0 \text{ if } (i, j) \in S, \quad u_{ij} = 0 \text{ if } (i, j) \in S, \quad \text{and} \quad r_{ij} = 0 \text{ if } (i, j) \notin S.$$

The factors L and U are unique and the splitting $A = LU - R$ is a regular splitting [Var60, MV77]. The idea of generating such approximate factorizations was considered by a number of people, one of the first of whom was Varga [Var60]. The idea became popular when it was used by Meijerink and van der Vorst to generate preconditioners for the conjugate gradient method and related iterations [MV77]. It has proved a successful technique in a range of applications and is now widely used with many variations. For example, instead of specifying the sparsity pattern of L , one might begin to compute the entries of the exact L -factor and set entries to 0 if they fall below some threshold (see, e.g., [Mun80]).

2. For a real symmetric positive definite matrix A arising from a standard finite difference or finite element approximation for a second order self-adjoint elliptic partial differential equation on a grid

with spacing h , the condition number of A is $O(h^{-2})$. When A is preconditioned using the incomplete Cholesky decomposition LL^T , where L has the same sparsity pattern as the lower triangle of A , the condition number of the preconditioned matrix $L^{-1}AL^{-T}$ is still $O(h^{-2})$, but the constant multiplying h^{-2} is smaller. When A is preconditioned using the modified incomplete Cholesky decomposition, the condition number of the preconditioned matrix is $O(h^{-1})$ [DKR68, Gus78].

3. For a general matrix A , the incomplete LU decomposition can be used as a preconditioner in a non-Hermitian matrix iteration such as GMRES, QMR, or BiCGSTAB. At each step of the preconditioned algorithm one must solve a linear system $Mz = r$. This is accomplished by first solving the lower triangular system $Ly = r$ and then solving the upper triangular system $Uz = y$.
4. One difficulty with incomplete Cholesky and incomplete LU decompositions is that the solution of the triangular systems may not parallelize well. In order to make better use of parallelism, sparse approximate inverses have been proposed as preconditioners. Here, a sparse matrix M^{-1} is constructed directly to approximate A^{-1} , and each step of the iteration method requires computation of a matrix-vector product $z = M^{-1}r$. For an excellent recent survey of all of these preconditioning methods see [Ben02].
5. Multigrid methods have the very desirable property that for many problems arising from elliptic PDEs the number of cycles required to reduce the error to a desired fixed level is independent of the grid size. This is in contrast to methods such as ICCG and MICCG (incomplete and modified incomplete Cholesky decomposition used as preconditioners in the CG algorithm). Early developers of multigrid methods include Fedorenko [Fed61] and later Brandt [Bra77]. A very readable and up-to-date introduction to the subject can be found in [BHM00].
6. Algebraic multigrid methods represent an attempt to use principles similar to those used for PDE problems on grids, when the origin of the problem is not necessarily known and only the matrix is available. An example is the AMG code by Ruge and Stüben [RS87]. The AMG method attempts to achieve mesh-independent convergence rates, just like standard multigrid methods, without making use of the underlying grid. A related class of preconditioners are *domain decomposition* methods. (See [QV99] and [SBG96] for recent surveys.)

41.5 Preconditioned Algorithms

Facts:

1. It is easy to modify the algorithms of the previous sections to use left preconditioning: Simply replace A by $M^{-1}A$ and b by $M^{-1}b$ wherever they appear. Since one need not actually compute M^{-1} , this is equivalent to solving linear systems with coefficient matrix M for the preconditioned quantities. For example, letting z_k denote the preconditioned residual $M^{-1}(b - Ax_k)$, the left-preconditioned BiCGSTAB algorithm is as follows:

Left-Preconditioned BiCGSTAB.

Given x_0 , compute $r_0 = b - Ax_0$, solve $Mz_0 = r_0$, and set $p_0 = z_0$.

Choose $\hat{z}_0$ such that $\langle z_0, \hat{z}_0 \rangle \neq 0$. For $k = 1, 2, \dots$,

Compute Ap_{k-1} and solve $Mq_{k-1} = Ap_{k-1}$.

Set $x_{k-1/2} = x_{k-1} + a_{k-1}p_{k-1}$, where $a_{k-1} = \frac{\langle z_{k-1}, \hat{z}_0 \rangle}{\langle q_{k-1}, \hat{z}_0 \rangle}$.

Compute $r_{k-1/2} = r_{k-1} - a_{k-1}Ap_{k-1}$ and $z_{k-1/2} = z_{k-1} - a_{k-1}q_{k-1}$.

Compute $Az_{k-1/2}$ and solve $Ms_{k-1/2} = Az_{k-1/2}$.

Set $x_k = x_{k-1/2} + \omega_k z_{k-1/2}$, where $\omega_k = \frac{\langle z_{k-1/2}, s_{k-1/2} \rangle}{\langle s_{k-1/2}, s_{k-1/2} \rangle}$.

Compute $r_k = r_{k-1/2} - \omega_k Az_{k-1/2}$ and $z_k = z_{k-1/2} - \omega_k s_{k-1/2}$.

Compute $p_k = z_k + b_k(p_{k-1} - \omega_k q_{k-1})$, where $b_k = \frac{a_{k-1}}{\omega_k} \frac{\langle z_k, \hat{z}_0 \rangle}{\langle z_{k-1}, \hat{z}_0 \rangle}$.

2. Right or Hermitian preconditioning requires a little more thought since we want to generate approximations $\mathbf{x}_k$ to the solution of the original linear system, not the modified one $AM^{-1}\mathbf{y} = \mathbf{b}$ or $L^{-1}AL^{-*}\mathbf{y} = L^{-1}\mathbf{b}$.

If the CG algorithm is applied directly to the problem $L^{-1}AL^{-*}\mathbf{y} = L^{-1}\mathbf{b}$, then the iterates satisfy

$$\begin{aligned} \mathbf{y}_k &= \mathbf{y}_{k-1} + a_{k-1}\hat{\mathbf{p}}_{k-1}, & a_{k-1} &= \frac{\langle \hat{\mathbf{r}}_{k-1}, \hat{\mathbf{r}}_{k-1} \rangle}{\langle \hat{\mathbf{p}}_{k-1}, L^{-1}AL^{-*}\hat{\mathbf{p}}_{k-1} \rangle}, \\ \hat{\mathbf{r}}_k &= \hat{\mathbf{r}}_{k-1} - a_{k-1}L^{-1}AL^{-*}\hat{\mathbf{p}}_{k-1}, \\ \hat{\mathbf{p}}_k &= \hat{\mathbf{r}}_k + b_{k-1}\hat{\mathbf{p}}_{k-1}, & b_{k-1} &= \frac{\langle \hat{\mathbf{r}}_k, \hat{\mathbf{r}}_k \rangle}{\langle \hat{\mathbf{r}}_{k-1}, \hat{\mathbf{r}}_{k-1} \rangle}. \end{aligned}$$

Defining

$$\mathbf{x}_k \equiv L^{-*}\mathbf{y}_k, \quad \mathbf{r}_k \equiv L\hat{\mathbf{r}}_k, \quad \mathbf{p}_k \equiv L^{-*}\hat{\mathbf{p}}_k,$$

we obtain the following preconditioned CG algorithm for $A\mathbf{x} = \mathbf{b}$:

Preconditioned Conjugate Gradient Method (PCG).

(For Hermitian Positive Definite Problems, with Hermitian Positive Definite Preconditioners)

Given an initial guess $\mathbf{x}_0$, compute $\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$ and solve

$M\mathbf{z}_0 = \mathbf{r}_0$. Set $\mathbf{p}_0 = \mathbf{z}_0$. For $k = 1, 2, \dots$,

Compute $A\mathbf{p}_{k-1}$.

Set $\mathbf{x}_k = \mathbf{x}_{k-1} + a_{k-1}\mathbf{p}_{k-1}$, where $a_{k-1} = \frac{\langle \mathbf{r}_{k-1}, \mathbf{z}_{k-1} \rangle}{\langle \mathbf{p}_{k-1}, A\mathbf{p}_{k-1} \rangle}$.

Compute $\mathbf{r}_k = \mathbf{r}_{k-1} - a_{k-1}A\mathbf{p}_{k-1}$.

Solve $M\mathbf{z}_k = \mathbf{r}_k$.

Set $\mathbf{p}_k = \mathbf{z}_k + b_{k-1}\mathbf{p}_{k-1}$, where $b_{k-1} = \frac{\langle \mathbf{r}_k, \mathbf{z}_k \rangle}{\langle \mathbf{r}_{k-1}, \mathbf{z}_{k-1} \rangle}$.

Applications:

1. Figure 41.4 shows the convergence (in terms of the 2-norm of the residual) of the ICCG algorithm (CG with incomplete Cholesky decomposition as a preconditioner, where the sparsity pattern of

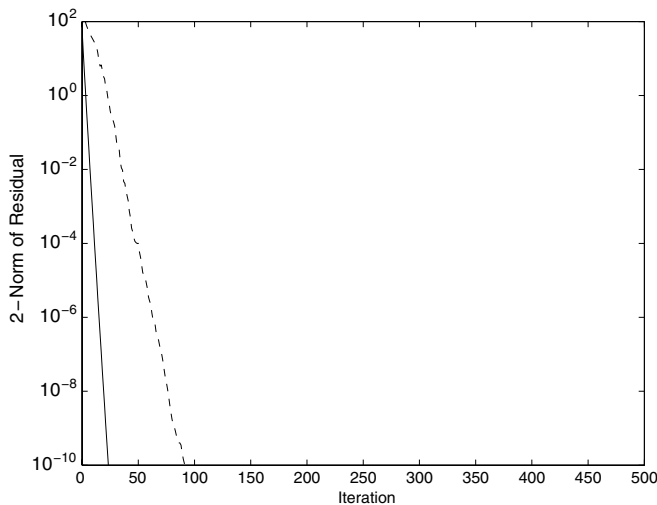


FIGURE 41.4 Convergence of ICCG (dashed) and multigrid (solid) for the problem given in the introduction with $a(x, y, z) = 1 + x + 3yz$, $h = 1/50$ (for ICCG), and $h = 1/64$ (for multigrid).

L was taken to match that of the lower triangle of A) and a multigrid method (using standard restriction and prolongation operators and the Gauss-Seidel algorithm for relaxation) for the same problem used in sections 2 and 3. The horizontal axis represents iterations for ICCG or cycles for the multigrid method. Each multigrid V-cycle costs about twice as much as an ICCG iteration, since it performs two triangular solves and two matrix vector multiplications on each coarse grid, while doing one of each on the fine grid. There is also a cost for the restriction and prolongation operations. The grid size was taken to be 64^3 for the multigrid method since this enabled easy formation of coarser grids by doubling h . The coarsest grid, on which the problem was solved directly, was taken to be of size $4 \times 4 \times 4$. It should be noted that the number of multigrid cycles can be reduced by using the multigrid procedure as a preconditioner for the CG algorithm (although this would require a different relaxation method in order to maintain symmetry). The only added expense is the cost of inner products in the CG algorithm.

41.6 Convergence Rates of CG and MINRES

In this section, we again let A and $\mathbf{b}$ denote the already preconditioned matrix and right-hand side vector, and we assume that A is Hermitian (and also positive definite for CG).

Facts:

1. The CG error vector and the MINRES residual vector at step k can be written in the form

$$\mathbf{e}_k = P_k^C(A)\mathbf{e}_0, \quad \mathbf{r}_k = P_k^M(A)\mathbf{r}_0,$$

where P_k^C and P_k^M are the k th degree polynomials with value 1 at the origin that minimize the A -norm of the error in the CG algorithm and the 2-norm of the residual in the MINRES algorithm, respectively. In other words, the error $\mathbf{e}_k$ in the CG approximation satisfies

$$\|\mathbf{e}_k\|_A = \min_{p_k} \|p_k(A)\mathbf{e}_0\|_A$$

and the residual $\mathbf{r}_k$ in the MINRES algorithm satisfies

$$\|\mathbf{r}_k\| = \min_{p_k} \|p_k(A)\mathbf{r}_0\|,$$

where the minimum is taken over all polynomials p_k of degree k or less with $p_k(0) = 1$.

2. Let an eigendecomposition of A be written as $A = U\Lambda U^*$, where U is a unitary matrix and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ is a diagonal matrix of eigenvalues. If A is positive definite, define $A^{1/2}$ to be $U\Lambda^{1/2}U^*$. Then the A -norm of a vector $\mathbf{v}$ is just the 2-norm of the vector $A^{1/2}\mathbf{v}$. The equalities in Fact 1 imply

$$\begin{aligned} \|\mathbf{e}_k\|_A &= \min_{p_k} \|A^{1/2} p_k(A)\mathbf{e}_0\| = \min_{p_k} \|p_k(A)A^{1/2}\mathbf{e}_0\| \\ &= \min_{p_k} \|U p_k(\Lambda)U^* A^{1/2}\mathbf{e}_0\| \leq \min_{p_k} \|p_k(\Lambda)\| \|\mathbf{e}_0\|_A, \end{aligned}$$

$$\|\mathbf{r}_k\| = \min_{p_k} \|U p_k(\Lambda)U^* \mathbf{r}_0\| \leq \min_{p_k} \|p_k(\Lambda)\| \|\mathbf{r}_0\|.$$

These bounds are *sharp*; that is, for each step k there is an initial vector for which equality holds [Gre79, GG94, Jou96], but the initial vector that gives equality at step k may be different from the one that results in equality at some other step j .

The problem of describing the convergence of these algorithms therefore reduces to one in approximation theory — How well can one approximate zero on the set of eigenvalues of A using a k th degree polynomial with value 1 at the origin? While there is no simple expression for the maximum value of the minimax polynomial on a discrete set of points, this minimax polynomial can be calculated if the eigenvalues of A are known and, more importantly, this sharp upper bound provides intuition as to what constitute “good” and “bad” eigenvalue distributions. Eigenvalues tightly clustered around a single point (away from the origin) are good, for instance, because the polynomial $(1 - z/c)^k$ is small in absolute value at all points near c . Widely spread eigenvalues, especially if they lie on both sides of the origin, are bad, because a low degree polynomial with value 1 at the origin cannot be small at a large number of such points.

3. Since one usually has only limited information about the eigenvalues of A , it is useful to have error bounds that involve only a few properties of the eigenvalues. For example, in the CG algorithm for Hermitian positive definite problems, knowing only the largest and smallest eigenvalues of A , one can obtain an error bound by considering the minimax polynomial on the *interval* from $\lambda_{\min}$ to $\lambda_{\max}$; i.e., the Chebyshev polynomial shifted to the interval and scaled to have value 1 at the origin. The result is

$$\frac{\|\mathbf{e}_k\|_A}{\|\mathbf{e}_0\|_A} \leq 2 \left[\left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k + \left(\frac{\sqrt{\kappa} + 1}{\sqrt{\kappa} - 1} \right)^k \right]^{-1} \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k,$$

where $\kappa = \lambda_{\max}/\lambda_{\min}$ is the ratio of largest to smallest eigenvalue of A .

If additional information is available about the interior eigenvalues of A , one often can improve on this estimate while maintaining a simpler expression than the sharp bound in Fact 2. Suppose, for example, that A has just a few eigenvalues that are much larger than the others, say, $\lambda_1 \leq \dots \leq \lambda_{n-\ell} \ll \lambda_{n-\ell+1} \leq \dots \leq \lambda_n$. Consider a polynomial p_k that is the product of a factor of degree ℓ that is zero at $\lambda_{n-\ell+1}, \dots, \lambda_n$ and the $(k - \ell)$ th degree scaled and shifted Chebyshev polynomial on the interval $[\lambda_1, \lambda_{n-\ell}]$:

$$p_k(x) = \left[T_{k-\ell} \left(\frac{2x - \lambda_{n-\ell} - \lambda_1}{\lambda_{n-\ell} - \lambda_1} \right) \right] / T_{k-\ell} \left(\frac{-\lambda_{n-\ell} - \lambda_1}{\lambda_{n-\ell} - \lambda_1} \right) \cdot \left(\prod_{i=n-\ell+1}^n \frac{\lambda_i - x}{\lambda_i} \right).$$

Since the second factor is zero at $\lambda_{n-\ell+1}, \dots, \lambda_n$ and less than one in absolute value at each of the other eigenvalues, the maximum absolute value of this polynomial on $\{\lambda_1, \dots, \lambda_n\}$ is less than the maximum absolute value of the first factor on $\{\lambda_1, \dots, \lambda_{n-\ell}\}$. It follows that

$$\frac{\|\mathbf{e}_k\|_A}{\|\mathbf{e}_0\|_A} \leq 2 \left(\frac{\sqrt{\kappa_{n-\ell}} - 1}{\sqrt{\kappa_{n-\ell}} + 1} \right)^{k-\ell}, \quad \kappa_{n-\ell} \equiv \frac{\lambda_{n-\ell}}{\lambda_1}.$$

Analogous results hold for the 2-norm of the residual in the MINRES algorithm applied to a Hermitian positive definite linear system. For estimates of the convergence rate of the MINRES algorithm applied to indefinite linear systems see, for example, [Fis96].

41.7 Convergence Rate of GMRES

Facts:

1. Like MINRES for Hermitian problems, the GMRES algorithm for general linear systems produces a residual at step k whose 2-norm satisfies $\|\mathbf{r}_k\| = \min_{p_k} \|p_k(A)\mathbf{r}_0\|$, where the minimum is over all k th degree polynomials p_k with $p_k(0) = 1$. To derive a bound on this expression that is

independent of the direction of $\mathbf{r}_0$, we could proceed as in the previous section by employing an eigendecomposition of A . To this end, assume that A is diagonalizable and let $A = V\Lambda V^{-1}$ be an eigendecomposition, where $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ is a diagonal matrix of eigenvalues and the columns of V are right eigenvectors of A . Then it follows that

$$\|\mathbf{r}_k\| = \min_{p_k} \|V p_k(\Lambda) V^{-1} \mathbf{r}_0\| \leq \kappa(V) \min_{p_k} \|p_k(\Lambda)\| \cdot \|\mathbf{r}_0\|,$$

where $\kappa(V) = \|V\| \cdot \|V^{-1}\|$ is the condition number of the eigenvector matrix V . We can assume that the columns of V have been scaled to make this condition number as small as possible. As in the Hermitian case, the polynomial that minimizes $\|V p_k(\Lambda) V^{-1} \mathbf{r}_0\|$ is not necessarily the one that minimizes $\|p_k(\Lambda)\|$, and it is not clear whether this bound is sharp. It turns out that if A is a normal matrix, then $\kappa(V) = 1$ and the bound is sharp [GG94, Jou96]. In this case, as in the Hermitian case, the problem of describing the convergence of GMRES reduces to a problem in approximation theory — How well can one approximate zero on the set of *complex* eigenvalues using a k th degree polynomial with value 1 at the origin?

If the matrix A is nonnormal but has a fairly well-conditioned eigenvector matrix V , then the above bound, while not necessarily sharp, gives a reasonable estimate of the actual size of the residual. In this case again, it is A 's eigenvalue distribution that essentially determines the behavior of GMRES.

2. In general, however, the behavior of GMRES cannot be determined from eigenvalues alone. In fact, it is shown in [GS94b] and [GPS96] that *any* nonincreasing sequence of residual norms can be obtained with the GMRES method applied to a problem whose coefficient matrix has any desired eigenvalues. Thus, for example, eigenvalues tightly clustered around 1 are not necessarily good for nonnormal matrices; one can construct a nonnormal matrix whose eigenvalues are equal to 1 or as tightly clustered around 1 as one might like, for which the GMRES algorithm makes no progress until step n (when it must find the exact solution).

The convergence behavior of the GMRES algorithm for nonnormal matrices is a topic of current research. The analysis must involve quantities other than the eigenvalues. Some partial results have been obtained in terms of the *field of values* [EES83, Eie93, BGT05], in terms of the ϵ -*pseudospectrum* [TE05], and in terms of the *polynomial numerical hull of degree k* [Gre02, Gre04].

41.8 Inexact Preconditioners and Finite Precision Arithmetic, Error Estimation and Stopping Criteria, Text and Reference Books

There are a number of topics of current or recent research that will not be covered in this article. Here we list a few of these with sample references.

Facts:

1. The effects of finite precision arithmetic on both the convergence rate and the ultimately attainable accuracy of iterative methods have been studied. Example papers include [DGR95, Gre89, Gre97b, GRS97, GS92, GS00, Pai76, Vor90], and [Woz80].
2. A related topic is inexact preconditioners. Suppose the preconditioning system $M\mathbf{z} = \mathbf{r}$ is solved inexactly, perhaps by using an iterative method inside the outer iteration for $A\mathbf{x} = \mathbf{b}$. How accurately should the preconditioning system be solved in order to obtain the best overall performance of the process? The answer is surprising. See [BF05, ES04], and [SS03] for recent discussions of this question.
3. Another related idea is the use of different preconditioners at different steps of the iteration, sometimes called *flexible* iterative methods. See, for example, [Saa93].

4. Finally, there is the important question of when an iterative method should be stopped. Sometimes one wishes to stop when the 2-norm of the error or the A -norm of the error in the CG method reaches a certain threshold. But one cannot compute these quantities directly. For discussions of estimating the A -norm of the error in the CG algorithm, as well as its connections to Gauss quadrature, see, for instance, [GM97, GS94, HS52], and [ST02]
5. A number of text and reference books on iterative methods are available. These include [Axe95, Gre97, Hac94, Saa03], and [Vor03]

References

- [Arn51] W.E. Arnoldi. The principal of minimized iterations in the solution of the matrix eigenvalue problem. *Q. Appl. Math.*, 9:17–29, 1951.
- [Axe95] O. Axelsson. *Iterative Solution Methods*. Cambridge University Press, Cambridge, 1995.
- [BGT05] B. Beckermann, S.A. Goreinov, and E.E. Tyrtyshnikov. Some remarks on the Elman estimate for GMRES. to appear.
- [Ben02] M. Benzi. Preconditioning techniques for large linear systems: A survey. *J. Comput. Phys.*, 182:418–477, 2002.
- [BF05] A. Bouras and V. Frayssé. Inexact matrix-vector products in Krylov methods for solving linear systems: A relaxation strategy. *SIAM J. Matrix Anal. Appl.*, 26:660–678, 2005.
- [Bra77] A. Brandt. Multi-level adaptive solutions to boundary-value problems. *Math. Comput.*, 31:333–390, 1977.
- [BHM00] W.L. Briggs, V.E. Henson, and S.F. McCormick. *A Multigrid Tutorial*. SIAM, Philadelphia, 2000.
- [DeS99] E. De Sturler. Truncation strategies for optimal Krylov subspace methods. *SIAM J. Numer. Anal.*, 36:864–889, 1999.
- [DGR95] J. Drkošová, A. Greenbaum, M. Rozložník, and Z. Strakoš. Numerical stability of the GMRES method. *BIT*, 3:309–330, 1995.
- [DKR68] T. Dupont, R.P. Kendall, and H.H. Rachford, Jr. An approximate factorization procedure for solving self-adjoint elliptic difference equations. *SIAM J. Numer. Anal.*, 5:559–573, 1968.
- [Eie93] M. Eiermann. Fields of values and iterative methods. *Lin. Alg. Appl.*, 180:167–197, 1993.
- [EES83] S. Eisenstat, H. Elman, and M. Schultz. Variational iterative methods for nonsymmetric systems of linear equations. *SIAM J. Numer. Anal.*, 20:345–357, 1983.
- [ES04] J. van den Eshof and G.L.G. Sleijpen. Inexact Krylov subspace methods for linear systems. *SIAM J. Matrix Anal. Appl.*, 26:125–153, 2004.
- [FM84] V. Faber and T. Manteuffel. Necessary and sufficient conditions for the existence of a conjugate gradient method. *SIAM J. Numer. Anal.*, 21:352–362, 1984.
- [Fed61] R. P. Fedorenko. A relaxation method for solving elliptic difference equations. *USSR Comput. Math. Math. Phys.*, 1:1092–1096, 1961.
- [Fis96] B. Fischer. *Polynomial Based Iteration Methods for Symmetric Linear Systems*. Wiley-Teubner, Leipzig, 1996.
- [Fle75] R. Fletcher. Conjugate gradient methods for indefinite systems, in *Proc. of the Dundee Biennial Conference on Numerical Analysis*, G.A. Watson, Ed. Springer-Verlag, Berlin/New York, 1975.
- [FN91] R.W. Freund and N.M. Nachtigal. QMR: A quasi-minimal residual method for non-Hermitian linear systems. *Numer. Math.*, 60:315–339, 1991.
- [GM97] G.H. Golub and G. Meurant. Matrices, moments, and quadrature II: How to compute the norm of the error in iterative methods. *BIT*, 37:687–705, 1997.
- [GS94] G.H. Golub and Z. Strakoš. Estimates in quadratic formulas. *Numer. Algorithms*, 8:241–268, 1994.
- [Gre79] A. Greenbaum. Comparison of splittings used with the conjugate gradient algorithm. *Numer. Math.*, 33:181–194, 1979.
- [Gre89] A. Greenbaum. Behavior of slightly perturbed Lanczos and conjugate gradient recurrences. *Lin. Alg. Appl.*, 113:7–63, 1989.
- [Gre97] A. Greenbaum. *Iterative Methods for Solving Linear Systems*. SIAM, Philadelphia, 1997.

- [Gre97b] A. Greenbaum. Estimating the attainable accuracy of recursively computed residual methods. *SIAM J. Matrix Anal. Appl.*, 18:535-551, 1997.
- [Gre02] A. Greenbaum. Generalizations of the field of values useful in the study of polynomial functions of a matrix. *Lin. Alg. Appl.*, 347:233-249, 2002.
- [Gre04] A. Greenbaum. Some theoretical results derived from polynomial numerical hulls of Jordan blocks. *Electron. Trans. Numer. Anal.*, 18:81-90, 2004.
- [GG94] A. Greenbaum and L. Gurvits. Max-min properties of matrix factor norms *SIAM J. Sci. Comput.*, 15:348-358, 1994.
- [GPS96] A. Greenbaum, V. Pták, and Z. Strakoš. Any non-increasing convergence curve is possible for GMRES *SIAM J. Matrix Anal. Appl.*, 17:465-469, 1996.
- [GRS97] A. Greenbaum, M. Rozložník, and Z. Strakoš. Numerical behavior of the modified Gram-Schmidt GMRES implementation. *BIT*, 37:706-719, 1997.
- [GS92] A. Greenbaum and Z. Strakoš. Behavior of finite precision Lanczos and conjugate gradient computations. *SIAM J. Matrix Anal. Appl.*, 13:121-137, 1992.
- [GS94b] A. Greenbaum and Z. Strakoš. Matrices that generate the same Krylov residual spaces. in *Recent Advances in Iterative Methods*, pp. 95-118, G. Golub, A. Greenbaum, and M. Luskin, Eds. Springer-Verlag, Berlin, New York, 1994.
- [Gus78] I. Gustafsson. A class of 1st order factorization methods. *BIT*, 18:142-156, 1978.
- [GS00] M. Gutknecht and Z. Strakoš. Accuracy of two three-term and three two-term recurrences for Krylov space solvers. *SIAM J. Matrix Anal. Appl.*, 22:213-229, 2000.
- [Hac94] W. Hackbusch. *Iterative Solution of Large Sparse Systems of Equations*. Springer-Verlag, Berlin, New York, 1994.
- [HS52] M.R. Hestenes and E. Stiefel. Methods of conjugate gradients for solving linear systems. *J. Res. Natl. Bur. Standards*, 49:409-435, 1952.
- [Jou96] W. Joubert. A robust GMRES-based adaptive polynomial preconditioning algorithm for nonsymmetric linear systems. *SIAM J. Sci. Comput.*, 15:427-439, 1994.
- [Lan50] C. Lanczos. An iteration method for the solution of the eigenvalue problem of linear differential and integral operators. *J. Res. Natl. Bur. Standards*, 45:255-282, 1950.
- [ML82] D.R. McCoy and E.W. Larsen. Unconditionally stable diffusion-synthetic acceleration methods for the slab geometry discrete ordinates equations, part II: Numerical results *Nuclear Sci. Engrg.*, 82:64-70, 1982.
- [MV77] J.A. Meijerink and H.A. van der Vorst. An iterative solution method for linear systems of which the coefficient matrix is a symmetric M-matrix. *Math. Comp.*, 31:148-162, 1977.
- [Mor95] R.B. Morgan. A restarted GMRES method augmented with eigenvectors. *SIAM J. Matrix Anal. Appl.*, 16:1154-1171, 1995.
- [Mun80] N. Munksgaard. Solving sparse symmetric sets of linear equations by preconditioned conjugate gradients. *ACM Trans. Math. Software*, 6:206-219, 1980.
- [Nac91] N. Nachtigal. A look-ahead variant of the Lanczos algorithm and its application to the quasi-minimal residual method for non-Hermitian linear systems. PhD dissertation, Massachusetts Institute of Technology, 1991.
- [Pai76] C.C. Paige. Error analysis of the Lanczos algorithm for tridiagonalizing a symmetric matrix. *J. Inst. Math. Appl.*, 18:341-349, 1976.
- [PTL85] B.N. Parlett, D.R. Taylor, and Z.A. Liu. A look-ahead Lanczos algorithm for unsymmetric matrices. *Math. Comp.*, 44:105-124, 1985.
- [QV99] A. Quarteroni and A. Valli. *Domain Decomposition Methods for Partial Differential Equations*. Clarendon, Oxford, 1999.
- [RS87] J.W. Ruge and K. Stüben. Algebraic multigrid, in *Multigrid Methods*, S.F. McCormick, Ed., p. 73. SIAM, Philadelphia, 1987.
- [Saa93] Y. Saad. A flexible inner-outer preconditioned GMRES algorithm. *SIAM J. Sci. Comput.*, 14:461-469, 1993.
- [Saa03] Y. Saad. *Iterative Methods for Sparse Linear Systems*, 2nd Edition. SIAM, Philadelphia, 2003.

- [SS86] Y. Saad and M.H. Schultz. GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM J. Sci. Stat. Comput.*, 7:856-869, 1986.
- [SS03] V. Simoncini and D.B. Szyld. Theory of inexact Krylov subspace methods and applications to scientific computing. *SIAM J. Sci. Comp.*, 28:454-477, 2003.
- [SBG96] B.F. Smith, P.E. Bjorstad, and W.D. Gropp. *Parallel Multilevel Methods for Elliptic Partial Differential Equations*. Cambridge Univ. Press, Cambridge/New York/Melbourne, 1996.
- [Son89] P. Sonneveld. CGS, a fast Lanczos-type solver for nonsymmetric linear systems. *SIAM J. Sci. Stat. Comput.*, 10:36-52, 1989.
- [ST02] Z. Strakoš and P. Tichý. On error estimation in the conjugate gradient method and why it works in finite precision computations. *Electron. Trans. Numer. Anal.*, 13:56-80, 2002.
- [TE05] L.N. Trefethen and M. Embree. *Spectra and Pseudospectra*, Princeton Univ. Press, Princeton, 2005.
- [Var60] R.S. Varga. Factorization and normalized iterative methods, in *Boundary Problems in Differential Equations*, R.E. Langer, Ed., pp. 121-142. 1960.
- [Vor90] H.A. van der Vorst. The convergence behavior of preconditioned CG and CG-S in the presence of rounding errors, in *Preconditioned Conjugate Gradient Methods*, O. Axelsson and L. Kolotilina, Eds., Lecture Notes in Mathematics 1457. Springer-Verlag, Berlin, 1990.
- [Vor92] H.A. van der Vorst. Bi-CGSTAB: A fast and smoothly converging variant of Bi-CG for the solution of nonsymmetric linear systems. *SIAM J. Sci. Comput.*, 13:631-644, 1992.
- [Vor03] H.A. van der Vorst. *Iterative Krylov Methods for Large Linear Systems*. Cambridge University Press, Cambridge, 2003.
- [Woz80] H. Wozniakowski. Roundoff error analysis of a new class of conjugate gradient algorithms. *Lin. Alg. Appl.*, 29:507-529, 1980.

Numerical Methods for Eigenvalues

42	Symmetric Matrix Eigenvalue Techniques	<i>Ivan Slapničar</i>	42-1
	Basic Methods • Tridiagonalization • Implicitly Shifted QR Method • Divide and Conquer Method • Bisection and Inverse Iteration • Multiple Relatively Robust Representations • Jacobi Method • Lanczos Method • Comparison of Methods		
43	Unsymmetric Matrix Eigenvalue Techniques	<i>David S. Watkins</i>	43-1
	The Generalized Eigenvalue Problem • Dense Matrix Techniques • Sparse Matrix Techniques		
44	The Implicitly Restarted Arnoldi Method	<i>D. C. Sorensen</i>	44-1
	Krylov Subspace Projection • The Arnoldi Factorization • Restarting the Arnoldi Process • Polynomial Restarting • Implicit Restarting • Convergence of IRAM • Convergence in Gap: Distance to a Subspace • The Generalized Eigenproblem • Krylov Methods with Spectral Transformations		
45	Computation of the Singular Value Decomposition	<i>Alan Kaylor Cline and Inderjit S. Dhillon</i>	45-1
	Singular Value Decomposition • Algorithms for the Singular Value Decomposition		
46	Computing Eigenvalues and Singular Values to High Relative Accuracy	<i>Zlatko Drmač</i>	46-1
	Accurate SVD and One-Sided Jacobi SVD Algorithm • Preconditioned Jacobi SVD Algorithm • Accurate SVD from a Rank Revealing Decomposition: Structured Matrices • Positive Definite Matrices • Accurate Eigenvalues of Symmetric Indefinite Matrices		

42

Symmetric Matrix Eigenvalue Techniques

42.1	Basic Methods	42-2
42.2	Tridiagonalization.....	42-5
42.3	Implicitly Shifted QR Method	42-9
42.4	Divide and Conquer Method	42-12
42.5	Bisection and Inverse Iteration.....	42-14
42.6	Multiple Relatively Robust Representations	42-15
42.7	Jacobi Method	42-17
42.8	Lanczos Method	42-19
42.9	Comparison of Methods	42-21
	References	42-22

Ivan Slapničar

University of Split, Croatia

The eigenvalue decomposition (EVD) is an infinite iterative procedure — finding eigenvalues is equivalent to finding zeros of the characteristic polynomial, and, by the results of Abel and Galois, there is no algebraic formula for roots of the polynomial of degree greater than four. However, the number of arithmetic operations required to compute EVD to some prescribed accuracy is also finite — EVD of a general symmetric matrix requires $O(n^3)$ operations, while for matrices with special structure this number can be smaller. For example, the EVD of a tridiagonal matrix can be computed in $O(n^2)$ operations (see Sections 42.5 and 42.6).

Basic methods for the symmetric eigenvalue computations are the power method, the inverse iteration method, and the QR iteration method (see Section 42.1). Since direct application of those methods to a general symmetric matrix requires $O(n^4)$ operations, the most commonly used algorithms consist of two steps: the given matrix is first reduced to tridiagonal form, followed by the computation of the EVD of the tridiagonal matrix by QR iteration, the divide and conquer method, bisection and inverse iteration, or the method of multiple relatively robust representations. Two other methods are the Jacobi method, which does not require tridiagonalization, and the Lanczos method, which computes only a part of the tridiagonal matrix.

Design of an efficient algorithm must take into account the target computer, the desired speed and accuracy, the specific goal (whether all or some eigenvalues and eigenvectors are desired), and the matrix size and structure (small or large, dense or sparse, tridiagonal, etc.). For example, if only some eigenvalues and eigenvectors are required, one can use the methods of Sections 42.5, 42.6, and 42.8. If high relative accuracy is desired and the matrix is positive definite, the Jacobi method is the method of choice. If the matrix is sparse, the Lanczos method should be used. We shall cover the most commonly used algorithms, like those which are implemented in LAPACK (see Chapter 75) and MATLAB® (see Chapter 71). The algorithms

provided in this chapter are intended to assist the reader in understanding the methods. Since the actual software is very complex, the reader is advised to use professional software in practice.

Efficient algorithms should be designed to use BLAS, and especially BLAS 3, as much as possible (see Chapter 74). The reasons are twofold: First, calling predefined standardized routines makes programs shorter and more easily readable, and second, processor vendors can optimize sets of standardized routines for their processor beyond the level given by compiler optimization. Examples of such optimized libraries are the *Intel Math Kernel Library* and *AMD Core Math Library*. Both libraries contain processor optimized BLAS, LAPACK, and FFT routines.

This chapter deals only with the computation of EVD of real symmetric matrices. The need to compute EVD of a complex Hermitian matrix (see Chapter 8) does not arise often in applications, and it is theoretically and numerically similar to the real symmetric case addressed here. All algorithms described in this chapter have their Hermitian counterparts (see e.g., [ABB99], [LSY98], and Chapters 71, 75, and 76).

The chapter is organized as follows: In Section 42.1, we describe basic methods for EVD computations. These methods are necessary to understand algorithms of Sections 42.3 to 42.6. In Section 42.2, we describe tridiagonalization by Householder reflections and Givens rotations. In Sections 42.3 to 42.6, we describe methods for computing the EVD of a tridiagonal matrix—QR iteration, the divide and conquer method, bisection and inverse iteration, and the method of multiple relatively robust representations, respectively. The Jacobi method is described in Section 42.7 and the Lanczos method is described in Section 42.8. For each method, we also describe the existing LAPACK or Matlab implementations. The respective timings of the methods are given in Section 42.9.

42.1 Basic Methods

Definitions:

The **eigenvalue decomposition** (EVD) of a real symmetric matrix $A = [a_{ij}]$ is given by $A = U\Lambda U^T$, where U is a $n \times n$ real orthonormal matrix, $U^T U = U U^T = I_n$, and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ is a real diagonal matrix.

The numbers λ_i are the **eigenvalues** of A , the columns $\mathbf{u}_i$, $i = 1, \dots, n$, of U are the **eigenvectors** of A , and $A\mathbf{u}_i = \lambda_i \mathbf{u}_i$, $i = 1, \dots, n$.

If $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$, we say that λ_1 is the **dominant eigenvalue**.

Deflation is a process of reducing the size of the matrix whose EVD is to be determined, given that one eigenvector is known (see Fact 4 below for details).

The **shifted matrix** of the matrix A is the matrix $A - \mu I$, where μ is the **shift**.

The simplest method for computing the EVD (also in the unsymmetric case) is the **power method**: given starting vector $\mathbf{x}_0$, the method computes the sequences

$$\mathbf{v}_k = \mathbf{x}_k^T A \mathbf{x}_k, \quad \mathbf{x}_{k+1} = A \mathbf{x}_k / \|A \mathbf{x}_k\|, \quad k = 0, 1, 2, \dots, \quad (42.1)$$

until convergence. Normalization of $\mathbf{x}_k$ can be performed in any norm and serves the numerical stability of the algorithm (avoiding overflow or underflow).

Inverse iteration is the power method applied to the inverse of a shifted matrix, starting from $\mathbf{x}_0$:

$$\mathbf{v}_k = \mathbf{x}_k^T A \mathbf{x}_k, \quad \mathbf{v}_{k+1} = (A - \mu I)^{-1} \mathbf{x}_k, \quad \mathbf{x}_{k+1} = \mathbf{v}_{k+1} / \|\mathbf{v}_{k+1}\|, \quad k = 0, 1, 2, \dots \quad (42.2)$$

Given starting $n \times p$ matrix X_0 with orthonormal columns, the **orthogonal iteration** (also **subspace iteration**) forms the sequence of matrices

$$Y_{k+1} = A X_k, \quad Y_{k+1} = X_{k+1} R_{k+1} \quad (\text{QR factorization}), \quad k = 0, 1, 2, \dots, \quad (42.3)$$

where $X_{k+1} R_{k+1}$ is the reduced QR factorization of Y_{k+1} (X_{k+1} is an $n \times p$ matrix with orthonormal columns and R_{k+1} is a upper triangular $p \times p$ matrix).

Starting from the matrix $A_0 = A$, the **QR iteration** forms the sequence of matrices

$$A_k = Q_k R_k \quad (\text{QR factorization}), \quad A_{k+1} = R_k Q_k, \quad k = 0, 1, 2, \dots \quad (42.4)$$

Given the shift μ , the **shifted QR iteration** forms the sequence of matrices

$$A_k - \mu I = Q_k R_k \quad (\text{QR factorization}), \quad A_{k+1} = R_k Q_k + \mu I, \quad k = 0, 1, 2, \dots \quad (42.5)$$

Facts:

The Facts 1 to 14 can be found in [GV96, §8.2], [Par80, §4, 5], [Ste01, §2.1, 2.2.1, 2.2.2], and [Dem97, §4].

1. If λ_1 is the dominant eigenvalue and if $\mathbf{x}_0$ is not orthogonal to $\mathbf{u}_1$, then in Equation 42.1 $v_k \rightarrow \lambda_1$ and $\mathbf{x}_k \rightarrow \mathbf{u}_1$. In other words, the power method converges to the dominant eigenvalue and its eigenvector.
2. The convergence of the power method is linear in the sense that

$$|\lambda_1 - v_k| = O\left(\left|\frac{\lambda_2}{\lambda_1}\right|^k\right), \quad \|\mathbf{u}_1 - \mathbf{x}_k\|_2 = O\left(\left|\frac{\lambda_2}{\lambda_1}\right|^k\right).$$

More precisely,

$$|\lambda_1 - v_k| \approx \left|\frac{c_2}{c_1}\right| \left|\frac{\lambda_2}{\lambda_1}\right|^k,$$

where c_i is the coefficient of the i -th eigenvector in the linear combination expressing the starting vector $\mathbf{x}_0$.

3. Since λ_1 is not readily available, the convergence is in practice determined using residuals. If $\|A\mathbf{x}_k - v_k\mathbf{x}_k\|_2 \leq \text{tol}$, where tol is a user prescribed stopping criterion, then $|\lambda_1 - v_k| \leq \text{tol}$.
4. After computing the dominant eigenpair, we can perform deflation to reduce the given EVD to the one of size $n - 1$. Let $Y = [\mathbf{u}_1 \quad X]$ be an orthogonal matrix. Then

$$\begin{bmatrix} \mathbf{u}_1 & X \end{bmatrix}^T A \begin{bmatrix} \mathbf{u}_1 & X \end{bmatrix} = \begin{bmatrix} \lambda_1 & 0 \\ 0 & A_1 \end{bmatrix},$$

where $A_1 = X^T A X$.

5. The EVD of the shifted matrix $A - \mu I$ is given by $U(\Lambda - \mu I)U^T$. Sometimes we can choose shift μ such that the shifted matrix $A - \mu I$ has better ratio between the dominant eigenvalue and the absolutely closest one, than the original matrix. In this case, applying the power method to the shifted matrix will speed up the convergence.
6. Inverse iteration requires solving the system of linear equations $(A - \mu I)\mathbf{v}_{k+1} = \mathbf{x}_k$ for $\mathbf{v}_{k+1}$ in each step. At the beginning, we must compute the LU factorization of $A - \mu I$, which requires $2n^3/3$ operations and in each subsequent step we must solve two triangular systems, which requires $2n^2$ operations.
7. If μ is very close to some eigenvalue of A , then the eigenvalues of the shifted matrix satisfy $|\lambda_1| \gg |\lambda_2| \geq \dots \geq |\lambda_n|$, so the convergence of the inverse iteration method is very fast.
8. If μ is very close to some eigenvalue of A , then the matrix $A - \mu I$ is nearly singular, so the solutions of linear systems may have large errors. However, these errors are almost entirely in the direction of the dominant eigenvector so the inverse iteration method is both fast and accurate.
9. We can further increase the speed of convergence of inverse iterations by substituting the shift μ with the Rayleigh quotient v_k in each step, at the cost of computing new LU factorization each time. See Chapter 8.2 for more information about the Rayleigh quotient.
10. If

$$|\lambda_1| \geq \dots \geq |\lambda_p| > |\lambda_{p+1}| \geq \dots \geq |\lambda_n|,$$

then the subspace iteration given in Equation 42.3 converges such that

$$X_k \rightarrow [\mathbf{u}_1, \dots, \mathbf{u}_p], \quad X_k^T A X_k \rightarrow \text{diag}(\lambda_1, \dots, \lambda_p),$$

at a speed which is proportional to $|\lambda_{p+1}/\lambda_p|^k$.

11. If $|\lambda_1| > |\lambda_2| > \dots > |\lambda_n|$, then the sequence of matrices A_k generated by the QR iteration given in Equation 42.4 converges to diagonal matrix Λ . However, this result is not of practical use, since the convergence may be very slow and each iteration requires $O(n^3)$ operations. Careful implementation, like the one described in section 42.3, is needed to construct an useful algorithm.
12. The QR iteration is equivalent to orthogonal iteration starting with the matrix $X_0 = I$. More precisely, the matrices X_k from Equation 42.3 and A_k from Equation 42.4 satisfy $X_k^T A X_k = A_k$.
13. Matrices A_k and A_{k+1} from Equations 42.4 and Equation 42.5 are orthogonally similar. In both cases

$$A_{k+1} = Q_k^T A_k Q_k.$$

14. The QR iteration method is essentially equivalent to the power method and the shifted QR iteration method is essentially equivalent to the inverse power method on the shifted matrix.
15. [Wil65, §3, 5, 6, 7] [TB97, §V] Let $U \Lambda U^T$ and $\tilde{U} \tilde{\Lambda} \tilde{U}^T$ be the exact and the computed EVDs of A , respectively, such that the diagonals of Λ and $\tilde{\Lambda}$ are in the same order. Numerical methods generally compute the EVD with the errors bounded by

$$|\lambda_i - \tilde{\lambda}_i| \leq \phi \epsilon \|A\|_2, \quad \|\mathbf{u}_i - \tilde{\mathbf{u}}_i\|_2 \leq \psi \epsilon \frac{\|A\|_2}{\min_{j \neq i} |\lambda_i - \tilde{\lambda}_j|},$$

where ϵ is machine precision and ϕ and ψ are slowly growing polynomial functions of n which depend upon the algorithm used (typically $O(n)$ or $O(n^2)$).

Examples:

1. The eigenvalue decomposition of the matrix

$$A = \begin{bmatrix} 4.5013 & 0.6122 & 2.1412 & 2.0390 \\ 0.6122 & 2.6210 & -0.4941 & -1.2164 \\ 2.1412 & -0.4941 & 1.1543 & -0.1590 \\ 2.0390 & -1.2164 & -0.1590 & -0.9429 \end{bmatrix}$$

computed by the MATLAB command `[U, Lambda]=eig(A)` is $A = U \Lambda U^T$ with (properly rounded to four decimal places)

$$U = \begin{bmatrix} -0.3697 & 0.2496 & 0.1003 & -0.8894 \\ 0.2810 & -0.0238 & 0.9593 & -0.0153 \\ 0.3059 & -0.8638 & -0.1172 & -0.3828 \\ 0.8311 & 0.4370 & -0.2366 & -0.2495 \end{bmatrix}, \quad \Lambda = \begin{bmatrix} -2.3197 & 0 & 0 & 0 \\ 0 & 0.6024 & 0 & 0 \\ 0 & 0 & 3.0454 & 0 \\ 0 & 0 & 0 & 6.0056 \end{bmatrix}.$$

2. Let A , U , and Λ be as in the Example 1, and set $\mathbf{x}_0 = [1 \ 1 \ 1 \ 1]^T$. The power method in Equation 42.1 gives $\mathbf{x}_6 = [0.8893 \ 0.0234 \ 0.3826 \ 0.2496]^T$. By setting $\mathbf{u}_1 = -U_{:,4}$ we have $\|\mathbf{u}_1 - \mathbf{x}_6\|_2 = 0.0081$. Here (Fact 2), $c_2 = 0.7058$, $c_1 = -1.5370$, and

$$\left| \frac{c_2}{c_1} \right| \left| \frac{\lambda_2}{\lambda_1} \right|^6 = 0.0078.$$

Similarly, $\|\mathbf{u}_1 - \mathbf{x}_{50}\|_2 = 1.3857 \cdot 10^{-15}$. However, for a different (bad) choice of the starting vector, $\mathbf{x}_0 = [0 \ 1 \ 0 \ 0]^T$, where $c_2 = 0.9593$ and $c_1 = -0.0153$, we have $\|\mathbf{u}_1 - \mathbf{x}_6\|_2 = 0.7956$.

3. The deflation matrix Y and the deflated matrix A_1 (Fact 4) for the above example are equal to (correctly rounded):

$$Y = \begin{bmatrix} -0.8894 & -0.0153 & -0.3828 & -0.2495 \\ -0.0153 & 0.9999 & -0.0031 & -0.0020 \\ -0.3828 & -0.0031 & 0.9224 & -0.0506 \\ -0.2495 & -0.0020 & -0.0506 & 0.9670 \end{bmatrix},$$

$$A_1 = \begin{bmatrix} 6.0056 & 0 & 0 & 0 \\ 0 & 2.6110 & -0.6379 & -1.3154 \\ 0 & -0.6379 & 0.2249 & -0.8952 \\ 0 & -1.3154 & -0.8952 & -1.5078 \end{bmatrix}.$$

4. Let A and $\mathbf{x}_0$ be as in Example 2. For the shift $\mu = 6$, the inverse iteration method in Equation 42.2 gives $\|\mathbf{u}_1 - \mathbf{x}_6\|_2 = 6.5187 \cdot 10^{-16}$, so the convergence is much faster than in Example 2 (Fact 7).
 5. Let A be as in Example 1. Applying six steps of the QR iteration in Equation 42.4 gives

$$A_6 = \begin{bmatrix} 6.0055 & -0.0050 & -0.0118 & -0.0000 \\ -0.0050 & 3.0270 & 0.3134 & 0.0002 \\ -0.0118 & 0.3134 & -2.3013 & -0.0017 \\ -0.0000 & 0.0002 & -0.0017 & 0.6024 \end{bmatrix}.$$

and applying six steps of the shifted QR iteration in Equation 42.5 with $\mu = 6$ gives

$$A_6 = \begin{bmatrix} -2.3123 & 0.1452 & -0.0215 & -0.0000 \\ 0.1452 & 0.6623 & 0.4005 & 0.0000 \\ -0.0215 & 0.4005 & 2.9781 & -0.0000 \\ 0.0000 & 0.0000 & 0.0000 & 6.0056 \end{bmatrix}.$$

In this case both methods converge. The convergence towards the matrix where the eigenvalue nearest to the shift can be deflated is faster for the shifted iterations.

42.2 Tridiagonalization

The QR iteration in Equation 42.4 in Section 42.1 and the shifted QR iteration in Equation 42.5 in Section 42.1 require $O(n^3)$ operations (one QR factorization) for each step, which makes these algorithms highly unpractical. However, if the starting matrix is tridiagonal, one step of these iterations requires only $O(n)$ operations. As a consequence, the practical algorithm consists of three steps:

1. Reduce A to tridiagonal form T by orthogonal similarities, $X^T A X = T$.
2. Compute the EVD of T , $T = Q \Lambda Q^T$.
3. Multiply $U = X Q$.

The EVD of A is then $A = U \Lambda U^T$. Reduction to tridiagonal form can be performed by using Householder reflectors or Givens rotations and it is a finite process requiring $O(n^3)$ operations. Reduction to tridiagonal form is a considerable compression of data since an EVD of T can be computed very quickly. The EVD of T can be efficiently computed by various methods such as QR iteration, the divide and conquer method (DC), bisection and inverse iteration, or the method of multiple relatively robust representations (MRRR). These methods are described in subsequent sections.

Facts:

All the following facts, except Fact 6, can be found in [Par80, §7], [TB97, pp. 196–201], [GV96, §8.3.1], [Ste01, pp. 158–162], and [Wil65, pp. 345–367].

1. Tridiagonal form is not unique (see Examples 1 and 2).
2. The reduction of A to tridiagonal matrix by Householder reflections is performed as follows. Let us partition A as

$$A = \left[\begin{array}{c|c} a_{11} & \mathbf{a}^T \\ \hline \mathbf{a} & B \end{array} \right].$$

Let H be the appropriate Householder reflection (see Chapter 38.4), that is,

$$\mathbf{v} = \mathbf{a} + \text{sign}(a_{21})\|\mathbf{a}\|_2 \mathbf{e}_1, \quad H = I - 2 \frac{\mathbf{v}\mathbf{v}^T}{\mathbf{v}^T \mathbf{v}},$$

and let

$$H_1 = \left[\begin{array}{c|c} 1 & \mathbf{0}^T \\ \hline \mathbf{0} & H \end{array} \right].$$

Then

$$H_1 A H_1 = \left[\begin{array}{c|c} a_{11} & \mathbf{a}^T H \\ \hline H \mathbf{a} & H B H \end{array} \right] = \left[\begin{array}{c|c} a_{11} & v \mathbf{e}_1^T \\ \hline v \mathbf{e}_1 & A_1 \end{array} \right], \quad v = -\text{sign}(a_{21})\|\mathbf{a}\|_2.$$

This step annihilates all elements in the first column below the first subdiagonal and all elements in the first row to the right of the first subdiagonal. Applying this procedure recursively yields the triangular matrix $T = X^T A X$, $X = H_1 H_2 \cdots H_{n-2}$.

3. H does not depend on the normalization of $\mathbf{v}$. The normalization $v_1 = 1$ is useful since $\mathbf{a}_{2:n}$ can be overwritten by $\mathbf{v}_{2:n}$ and v_1 does not need to be stored.
4. Forming H explicitly and then computing $A_1 = H B H$ requires $O(n^3)$ operations, which would ultimately yield an $O(n^4)$ algorithm. However, we do not need to form the matrix H explicitly — given $\mathbf{v}$, we can overwrite B with $H B H$ in just $O(n^2)$ operations by using one matrix-vector multiplication and two rank-one updates.
5. The entire tridiagonalization algorithm is as follows:

Algorithm 1: Tridiagonalization by Householder reflections

Input: real symmetric $n \times n$ matrix A

Output: the main diagonal and sub- and superdiagonal of A are overwritten by T ,
the Householder vectors are stored in the lower triangular part of A
below the first subdiagonal

```

for  $j = 1 : n - 2$ 
     $\mu = \text{sign}(a_{j+1,j})\|A_{j+1:n,j}\|_2$ 
    if  $\mu \neq 0$ , then
         $\beta = a_{j+1,j} + \mu$ 
         $\mathbf{v}_{j+2:n} = A_{j+2:n,j}/\beta$ 
    endif
     $a_{j+1,j} = -\mu$ 
     $a_{j,j+1} = -\mu$ 
     $v_{j+1} = 1$ 
     $\gamma = -2/\mathbf{v}_{j+1:n}^T \mathbf{v}_{j+1:n}$ 
     $\mathbf{w} = \gamma A_{j+1:n,j+1:n} \mathbf{v}_{j+1:n}$ 
     $\mathbf{q} = \mathbf{w} + \frac{1}{2} \gamma \mathbf{v}_{j+1:n} (\mathbf{v}_{j+1:n}^T \mathbf{w})$ 
     $A_{j+1:n,j+1:n} = A_{j+1:n,j+1:n} + \mathbf{v}_{j+1:n} \mathbf{q}^T + \mathbf{q} \mathbf{v}_{j+1:n}^T$ 
     $A_{j+2:n,j} = \mathbf{v}_{j+2:n}$ 
endfor

```

6. [DHS89] When symmetry is exploited in performing rank-2 update, Algorithm 1 requires $4n^3/3$ operations. Another important enhancement is the derivation of the block-version of the algorithm. Instead of performing rank-2 update on B , thus obtaining A_1 , we can accumulate p transformations and perform rank- $2p$ update. In the first p steps, the algorithm is modified to update only columns and rows $1, \dots, p$, which are needed to compute the first p Householder vectors. Then the matrix A is updated by $A - UV^T - VU^T$, where U and V are $n \times p$ matrices. This algorithm is rich in matrix-matrix multiplications (roughly one half of the operations is performed using BLAS 3 routines), but it requires extra workspace for U and V .
7. If the matrix X is needed explicitly, it can be computed from the stored Householder vectors by Algorithm 2. In order to minimize the operation count, the computation starts from the smallest matrix and the size is gradually increased, that is, the algorithm computes the sequence of matrices

$$H_{n-2}, \quad H_{n-3}H_{n-2}, \dots, \quad X = H_1 \cdots H_{n-2}.$$

A column-oriented version is possible as well, and the operation count in both cases is $4n^3/3$. If the Householder matrices H_i are accumulated in the order in which they are generated, the operation count is $2n^3$.

Algorithm 2: Computation of the tridiagonalizing matrix X

Input: output from Algorithm 1

Output: matrix X such that $X^TAX = T$, where A is the input of Algorithm 1 and T is tridiagonal.

$$X = I_n$$

for $j = n - 2 : -1 : 1$

$$v_{j+1} = 1$$

$$\mathbf{v}_{j+2:n} = A_{j+2:n,j}$$

$$\gamma = -2/\mathbf{v}_{j+1:n}^T \mathbf{v}_{j+1:n}$$

$$\mathbf{w} = \gamma X_{j+1:n,j+1:n}^T \mathbf{v}_{j+1:n}$$

$$X_{j+1:n,j+1:n} = X_{j+1:n,j+1:n} + \mathbf{v}_{j+1:n} \mathbf{w}^T$$

endfor

8. The error bounds for Algorithms 1 and 2 are as follows: The matrix $\tilde{T}$ computed by Algorithm 1 is equal to the matrix, which would be obtained by exact tridiagonalization of some perturbed matrix $A + E$ (backward error), where $\|E\|_2 \leq \psi \epsilon \|A\|_2$ and ψ is a slowly increasing function of n . The matrix $\tilde{X}$ computed by Algorithm 2 satisfies $\tilde{X} = X + F$, where $\|F\|_2 \leq \phi \epsilon$ and ϕ is a slowly increasing function of n .
9. Givens rotation parameters c and s are computed as in Fact 5 of Section 38.4. Tridiagonalization by Givens rotations is performed as follows:

Algorithm 3: Tridiagonalization by Givens rotations

Input: real symmetric $n \times n$ matrix A

Output: the matrix X such that $X^TAX = T$ is tridiagonal, main diagonal and sub- and superdiagonal of A are overwritten by T

$$X = I_n$$

for $j = 1 : n - 2$

for $i = j + 2 : n$

$$\text{set } x = a_{j+1,j} \text{ and } y = a_{i,j}$$

$$\text{compute } G = \begin{bmatrix} c & s \\ -s & c \end{bmatrix} \text{ via Fact 5 of Section 38.4}$$

Algorithm 3: Tridiagonalization by Givens rotations (Continued)
$$\begin{bmatrix} A_{j+1,j:n} \\ A_{i,j:n} \end{bmatrix} = G \begin{bmatrix} A_{j+1,j:n} \\ A_{i,j:n} \end{bmatrix}$$

$$\begin{bmatrix} A_{j:n,j+1} & A_{j:n,i} \end{bmatrix} = \begin{bmatrix} A_{j:n,j+1} & A_{j:n,i} \end{bmatrix} G^T$$

$$\begin{bmatrix} X_{1:n,j+1} & X_{1:n,i} \end{bmatrix} = \begin{bmatrix} X_{1:n,j+1} & X_{1:n,i} \end{bmatrix} G^T$$

endfor

endfor

10. Algorithm 3 requires $(n-1)(n-2)/2$ plane rotations, which amounts to $4n^3$ operations if symmetry is properly exploited. The operation count is reduced to $8n^3/3$ if fast rotations are used. Fast rotations are obtained by factoring out absolutely larger of c and s from G .
11. The Givens rotations in Algorithm 3 can be performed in different orderings. For example, the elements in the first column and row can be annihilated by rotations in the planes $(n-1, n)$, $(n-2, n-1)$, $\dots$ $(2, 3)$. Since Givens rotations act more selectively than Householder reflectors, they can be useful if A has some special structure. For example, Givens rotations are used to efficiently tridiagonalize symmetric band matrices (see Example 4).
12. Error bounds for Algorithm 3 are the same as the ones for Algorithms 1 and 2 (Fact 8), but with slightly different functions ψ and ϕ .

Examples:

1. Algorithms 1 and 2 applied to the matrix A from Example 1 in Section 42.1 give

$$T = \begin{bmatrix} 4.5013 & -3.0194 & 0 & 0 \\ -3.0194 & -0.3692 & 1.2804 & 0 \\ 0 & 1.2804 & 0.5243 & -0.9303 \\ 0 & 0 & -0.9303 & 2.6774 \end{bmatrix},$$

$$X = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -0.2028 & 0.4417 & -0.8740 \\ 0 & -0.7091 & -0.6817 & -0.1800 \\ 0 & -0.6753 & 0.5833 & 0.4514 \end{bmatrix}.$$

2. Tridiagonalization is implemented in the MATLAB function $T = \text{hess}(A)$ ($[X, T] = \text{hess}(A)$ if X is to be computed, as well). In fact, the function `hess` is more general and it computes the Hessenberg form of a general square matrix. For the same matrix A as above, the matrices T and X computed by `hess` are:

$$T = \begin{bmatrix} 2.6562 & 1.3287 & 0 & 0 \\ 1.3287 & 2.4407 & 2.4716 & 0 \\ 0 & 2.4716 & 3.1798 & 2.3796 \\ 0 & 0 & 2.3796 & -0.9429 \end{bmatrix}, \quad X = \begin{bmatrix} 0.4369 & 0.2737 & 0.8569 & 0 \\ 0.7889 & 0.3412 & -0.5112 & 0 \\ -0.4322 & 0.8993 & -0.0668 & 0 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}.$$

3. The block version of tridiagonal reduction is implemented in the LAPACK subroutine `DSYTRD` (file `dsytrd.f`). The computation of X is implemented in the subroutine `DORGTR`. The size of the required extra workspace (in elements) is $lwork = nb * n$, where nb is the optimal block size (here, $nb = 64$), and it is determined automatically by the subroutines. The timings are given in Section 42.9.

4. Computation of Givens rotation in Algorithm 3 is implemented in the MATLAB functions `planerot` and `givens`, BLAS 1 subroutine `DROTG`, and LAPACK subroutine `DLARTG`. These implementations avoid unnecessary overflow or underflow by appropriately scaling x and y . Plane rotations (multiplications with G) are implemented in the BLAS 1 subroutine `DROT`. LAPACK subroutines `DLAR2V`, `DLARGV`, and `DLARTV` generate and apply multiple plane rotations. LAPACK subroutine `DSBTRD` tridiagonalizes a symmetric band matrix by using Givens rotations.

42.3 Implicitly Shifted QR Method

This method is named after the fact that, for a tridiagonal matrix, each step of the shifted QR iterations given by Equation 42.5 in Section 42.1 can be elegantly implemented without explicitly computing the shifted matrix $A_k - \mu I$.

Definitions:

Wilkinson's shift μ is the eigenvalue of the bottom right 2×2 submatrix of T , which is closer to $t_{n,n}$.

Facts:

The following facts can be found in [GV96, pp. 417–422], [Ste01, pp. 163–171], [TB97, pp. 211–224], [Par80, §8], [Dem97, §5.3.1], and [Wil65, §8.50, 8.54].

$T = [t_{ij}]$ is a real symmetric tridiagonal matrix of order n and $T = QAQ^T$ is its EVD.

1. The stable formula for the Wilkinson's shift is

$$\mu = t_{n,n} - \frac{t_{n,n-1}^2}{\tau + \text{sign}(\tau)\sqrt{\tau^2 + t_{n,n-1}^2}}, \quad \tau = \frac{t_{n-1,n-1} - t_{n,n}}{2}.$$

2. The following recursive function implements the implicitly shifted QR method given by Equation 42.5:

Algorithm 4: Implicitly shifted QR method for tridiagonal matrices

Input: real symmetric tridiagonal $n \times n$ matrix T

Output: the diagonal of T is overwritten by its eigenvalues

function $T = \text{QR_iteration}(T)$

repeat % one sweep

 compute a suitable shift μ

 set $x = t_{11} - \mu$ and $y = t_{21}$

 compute $G = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}$ via Fact 5 of Chapter 38.4

$\begin{bmatrix} T_{1,1:3} \\ T_{2,1:3} \end{bmatrix} = G \begin{bmatrix} T_{1,1:3} \\ T_{2,1:3} \end{bmatrix}$

$\begin{bmatrix} T_{1:3,1} & T_{1:3,2} \end{bmatrix} = \begin{bmatrix} T_{1:3,1} & T_{1:3,2} \end{bmatrix} G^T$

for $i = 2 : n - 1$

 set $x = t_{i,i-1}$ and $y = t_{i+1,i-1}$

 compute $G = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}$ via Fact 5 of Section 38.4

Algorithm 4: Implicitly shifted QR method for tridiagonal matrices (Continued)

$$\begin{bmatrix} T_{i,i-1:i+2} \\ T_{i+1,i-1:i+2} \end{bmatrix} = G \begin{bmatrix} T_{i,i-1:i+2} \\ T_{i+1,i-1:i+2} \end{bmatrix}$$

$$\begin{bmatrix} T_{i-1:i+2,i} & T_{i-1:i+2,i+1} \end{bmatrix} = \begin{bmatrix} T_{i-1:i+2,i} & T_{i-1:i+2,i+1} \end{bmatrix} G^T$$

endfor

until $|t_{i,i+1}| \leq \epsilon \sqrt{|t_{i,i} \cdot t_{i+1,i+1}|}$ for some i % deflation

set $t_{i+1,i} = 0$ and $t_{i,i+1} = 0$

$T_{1:i,1:i} = \text{QR_iteration}(T_{1:i,1:i})$

$T_{i+1:n,i+1:n} = \text{QR_iteration}(T_{i+1:n,i+1:n})$

3. Wilkinson's shift (Fact 1) is the most commonly used shift. With Wilkinson's shift, the algorithm always converges in the sense that $t_{n-1,n} \rightarrow 0$. The convergence is quadratic, that is, $||[T_{k+1}]_{n-1,n}| \leq c|[T_k]_{n-1,n}|^2$ for some constant c , where T_k is the matrix after k -th sweep. Even more, the convergence is usually cubic. However, it can also happen that some $t_{i,i+1}$, $i \neq n-1$, becomes sufficiently small before $t_{n-1,n}$, so the practical program has to check for deflation at each step.
4. The plane rotation parameters at the start of the sweep are computed as if the shifted matrix $T - \mu I$ has been formed. Since the rotation is applied to the original T and not to $T - \mu I$, this creates new nonzero elements at the positions (3,1) and (1,3), the so-called **bulge**. The subsequent rotations simply chase the bulge out of the lower right corner of the matrix. The rotation in the (2,3) plane sets the elements (3,1) and (1,3) back to zero, but it generates two new nonzero elements at positions (4,2) and (2,4); the rotation in the (3,4) plane sets the elements (4,2) and (2,4) back to zero, but it generates two new nonzero elements at positions (5,3) and (3,5), etc. The procedure is illustrated in Figure 42.1: "x" denotes the elements that are transformed by the current plane rotation, "*" denotes the newly generated nonzero elements (the bulge), and 0 denotes the zeros that are reintroduced by the current plane rotation.

The effect of this procedure is the following. At the end of the first sweep, the resulting matrix T_1 is equal to the the matrix that would have been obtained by factorizing $T - \mu I = QR$ and computing $T_1 = RQ + \mu I$ as in Equation 42.5.

5. Since the convergence of Algorithm 4 is quadratic (or even cubic), an eigenvalue is isolated after just a few steps, which requires $O(n)$ operations. This means that $O(n^2)$ operations are needed to compute all eigenvalues.
6. If the eigenvector matrix Q is desired, the plane rotations need to be accumulated similarly to the accumulation of X in Algorithm 3. This accumulation requires $O(n^3)$ operations (see Example 2 below and Fact 5 in Section 42.9). Another, usually faster, algorithm to compute Q is given in Fact 9 in Section 42.9.

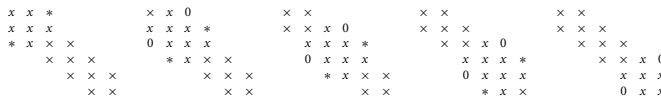


FIGURE 42.1 Chasing the bulge in one sweep of the implicit QR iteration for $n = 6$.

7. The computed eigenvalue decomposition $T = Q\Lambda Q^T$ satisfies the error bounds from Fact 15 in section 42.1 with A replaced by T and U replaced by Q . The deflation criterion implies $|t_{i,i+1}| \leq \epsilon \|T\|_F$, which is within these bounds.
8. Combining Algorithms 1, 2, and 4 we get the the following algorithm:

Algorithm 5: Real symmetric eigenvalue decomposition

Input: real symmetric $n \times n$ matrix A

Output: eigenvalue matrix Λ and, optionally, eigenvector matrix U of A

if only eigenvalues are required, then

 Compute T by Algorithm 1

$T = \text{QR_iteration}(T)$ % Algorithm 4

$\Lambda = \text{diag}(T)$

else

 Compute T by Algorithm 1

 Compute X by Algorithm 2

$T = \text{QR_iteration}(T)$ % with rotations accumulated in Q

$\Lambda = \text{diag}(T)$

$U = XQ$

endif

9. The EVD computed by Algorithm 5 satisfies the error bounds given in Fact 15 in section 42.1. However, the algorithm tends to perform better on matrices, which are graded downwards, that is, on matrices that exhibit systematic decrease in the size of the matrix elements as we move along the diagonal. For such matrices the tiny eigenvalues can usually be computed with higher relative accuracy (although counterexamples can be easily constructed). If the tiny eigenvalues are of interest, it should be checked whether there exists a symmetric permutation that moves larger elements to the upper left corner, thus converting the given matrix to the one that is graded downwards.

Examples:

1. For the matrix T from Example 1 in section 42.2, after one sweep of Algorithm 4, we have

$$T = \begin{bmatrix} 2.9561 & 3.9469 & 0 & 0 \\ 3.9469 & 0.8069 & -0.7032 & 0 \\ 0 & -0.7032 & 0.5253 & 0.0091 \\ 0 & 0 & 0.0091 & 3.0454 \end{bmatrix}.$$

2. Algorithm 4 is implemented in the LAPACK subroutine DSTEQR. This routine can compute just the eigenvalues, or both eigenvalues and eigenvectors. To avoid double indices, the diagonal and subdiagonal entries of T are stored in one dimensional vectors, $d_i = T_{ii}$ and $e_i = T_{i+1,i}$, respectively. The timings are given in Section 42.9.
3. Algorithm 5 is implemented in the Matlab routine `eig`. The command `Lambda = eig(A)` returns only the eigenvalues, `[U, Lambda] = eig(A)` returns the eigenvalues and the eigenvectors (see Example 1 in Section 42.1).
4. The LAPACK implementation of Algorithm 5 is given in the subroutine DSYEV. To compute only eigenvalues, DSYEV calls DSYTRD and DSTEQR without eigenvector option. To compute both eigenvalues and eigenvectors, DSYEV calls DSYTRD, DORGTR, and DSTEQR with the eigenvector option. The timings are given in Section 42.9.

42.4 Divide and Conquer Method

This is currently the fastest method for computing the EVD of a real symmetric tridiagonal matrix T . It is based on splitting the given tridiagonal matrix into two matrices, then computing the EVDs of the smaller matrices and computing the final EVD from the two EVDs. The method was first introduced in [Cup81], but numerically stable and efficient implementation was first derived in [GE95].

Facts:

The following facts can be found in [Dem97, pp. 216–228], [Ste01, pp. 171–185], and [GE95].

$T = [t_{ij}]$ is a real symmetric tridiagonal matrix of order n and $T = U \Lambda U^T$ is its EVD.

1. Let T be partitioned as

$$T = \left[\begin{array}{ccc|ccc} d_1 & e_1 & & & & \\ e_1 & d_2 & e_2 & & & \\ & & \ddots & \ddots & \ddots & \\ & & & e_{k-1} & d_k & \\ \hline & & & e_k & & \\ & & & d_{k+1} & e_{k+1} & \\ & & & & \ddots & \ddots \\ & & & & e_{n-2} & d_{n-1} & e_{n-1} \\ & & & & & e_{n-1} & d_n \end{array} \right] \equiv \begin{bmatrix} T_1 & e_k \mathbf{e}_k \mathbf{e}_1^T \\ e_k \mathbf{e}_1 \mathbf{e}_k^T & T_2 \end{bmatrix}.$$

We assume that T is unreduced, that is, $e_i \neq 0$ for all i . Further, we assume that $e_i > 0$ for all i , which can be easily be attained by diagonal similarity with a diagonal matrix of signs (see Example 1 below). Let

$$\hat{T}_1 = T_1 - e_k \mathbf{e}_k \mathbf{e}_k^T, \quad \hat{T}_2 = T_2 - e_k \mathbf{e}_1 \mathbf{e}_1^T. \quad (42.6)$$

In other words, $\hat{T}_1$ is equal to T_1 except that d_k is replaced by $d_k - e_k$, and $\hat{T}_2$ is equal to T_2 except that d_{k+1} is replaced by $d_{k+1} - e_k$.

Let $\hat{T}_i = \hat{U}_i \hat{\Lambda}_i \hat{U}_i^T$, $i = 1, 2$, be the respective EVDs and let $\mathbf{v} = \begin{bmatrix} \hat{U}_1^T \mathbf{e}_k \\ \hat{U}_2^T \mathbf{e}_1 \end{bmatrix}$ ($\mathbf{v}$ consists of the last column of $\hat{U}_1^T$ and the first column of $\hat{U}_2^T$). Set $\hat{U} = \hat{U}_1 \oplus \hat{U}_2$ and $\hat{\Lambda} = \hat{\Lambda}_1 \oplus \hat{\Lambda}_2$. Then

$$T = \begin{bmatrix} \hat{U}_1 & \\ & \hat{U}_2 \end{bmatrix} \left[\begin{bmatrix} \hat{\Lambda}_1 & \\ & \hat{\Lambda}_2 \end{bmatrix} + e_k \mathbf{v} \mathbf{v}^T \right] \begin{bmatrix} \hat{U}_1^T & \\ & \hat{U}_2^T \end{bmatrix} = \hat{U} (\hat{\Lambda} + e_k \mathbf{v} \mathbf{v}^T) \hat{U}^T. \quad (42.7)$$

If

$$\hat{\Lambda} + e_k \mathbf{v} \mathbf{v}^T = X \Lambda X^T$$

is the EVD of the rank-one modification of the diagonal matrix $\hat{\Lambda}$, then $T = U \Lambda U^T$, where $U = \hat{U} X$ is the EVD of T . Thus, the original tridiagonal eigenvalue problem is reduced to two smaller tridiagonal eigenvalue problems and one eigenvalue problem for the rank-one update of a diagonal matrix.

2. If the matrix $\hat{\Lambda} + e_k \mathbf{v} \mathbf{v}^T$ is permuted such that $\hat{\lambda}_1 \geq \dots \geq \hat{\lambda}_n$, then λ_i and $\hat{\lambda}_i$ are interlaced, that is,

$$\lambda_1 \geq \hat{\lambda}_1 \geq \lambda_2 \geq \hat{\lambda}_2 \geq \dots \geq \lambda_{n-1} \geq \hat{\lambda}_{n-1} \geq \lambda_n \geq \hat{\lambda}_n.$$

Moreover, if $\hat{\lambda}_{i-1} = \hat{\lambda}_i$ for some i , then one eigenvalue is obviously known exactly, that is, $\lambda_i = \hat{\lambda}_i$. In this case, λ_i can be deflated by applying to $\hat{\Lambda} + e_k \mathbf{v} \mathbf{v}^T$ a plane rotation in the $(i-1, i)$ plane, where the Givens rotation parameters c and s are computed from v_{i-1} and v_i as in Fact 5 of Section 38.4.

3. If all $\hat{\lambda}_i$ are different, then the eigenvalues λ_i of $\hat{\Lambda} + e_k \mathbf{v}\mathbf{v}^T$ are solutions of the so-called secular equation,

$$1 + e_k \sum_{i=1}^n \frac{v_i^2}{\hat{\lambda}_i - \lambda} = 0.$$

The eigenvalues can be computed by bisection, or by some faster zero finder of the Newton type, and they need to be computed as accurately as possible.

4. Once the eigenvalues λ_i of $\hat{\Lambda} + e_k \mathbf{v}\mathbf{v}^T$ are known, the corresponding eigenvectors are

$$\mathbf{x}_i = (\hat{\Lambda} - \lambda_i I)^{-1} \mathbf{v}.$$

5. Each λ_i and $\mathbf{x}_i$ in Facts 3 and 4 is computed in $O(n)$ operations, respectively, so the overall computational cost for computing the EVD of $\hat{\Lambda} + e_k \mathbf{v}\mathbf{v}^T$ is $O(n^2)$.
6. The accuracy of the computed EVD is given by Fact 15 in section 42.1. However, if some eigenvalues are too close, they may not be computed with sufficient relative accuracy. As a consequence, the eigenvectors computed by using Fact 4 may not be sufficiently orthogonal. One remedy to this problem is to solve the secular equation from Fact 3 in double of the working precision. A better remedy is based on the solution of the following inverse eigenvalue problem. If $\hat{\lambda}_1 > \dots > \hat{\lambda}_n$ and $\lambda_1 > \hat{\lambda}_1 > \lambda_2 > \hat{\lambda}_2 > \dots > \lambda_{n-1} > \hat{\lambda}_{n-1} > \lambda_n > \hat{\lambda}_n$, then λ_i are the exact eigenvalues of the matrix $\hat{\Lambda} + e_k \hat{\mathbf{v}}\hat{\mathbf{v}}^T$, where

$$\hat{v}_i = \text{sign } v_i \sqrt{\frac{\prod_{j=1}^n (\lambda_j - \hat{\lambda}_i)}{\prod_{j=1, j \neq i}^n (\hat{\lambda}_j - \hat{\lambda}_i)}}.$$

Instead of computing $\mathbf{x}_i$ according to Fact 4, we compute $\hat{\mathbf{x}}_i = (\hat{\Lambda} - \lambda_i I)^{-1} \hat{\mathbf{v}}$. The eigenvector matrix of T is now computed as $U = \hat{U} \hat{X}$, where $\hat{X} = [\hat{\mathbf{x}}_1 \dots \hat{\mathbf{x}}_n]$, instead of $U = \hat{U} X$ as in Fact 1. See also Fact 8.

7. The algorithm for the divide and conquer method is the following:

Algorithm 6: Divide and conquer method

Input: real symmetric tridiagonal $n \times n$ matrix T with $t_{i-1,i} > 0$ for all i

Output: eigenvalue matrix Λ and eigenvector matrix U of T

function $(\Lambda, U) = \text{Divide_and_Conquer}(T)$

if $n = 1$, **then**

$U = 1$

$\Lambda = T$

else

$k = \text{floor}(n/2)$

 form $\hat{T}_1$ and $\hat{T}_2$ as in Equation 42.6 in Fact 1

$(\hat{\Lambda}_1, \hat{U}_1) = \text{Divide_and_Conquer}(\hat{T}_1)$

$(\hat{\Lambda}_2, \hat{U}_2) = \text{Divide_and_Conquer}(\hat{T}_2)$

 form $\hat{\Lambda} + e_k \mathbf{v}\mathbf{v}^T$ as in Equation 42.7 in Fact 1

 compute the eigenvalues λ_i via Fact 3

 compute $\hat{\mathbf{v}}$ via Fact 6

$\hat{\mathbf{x}}_i = (\hat{\Lambda} - \lambda_i I)^{-1} \hat{\mathbf{v}}$

$U = \begin{bmatrix} \hat{U}_1 \\ \hat{U}_2 \end{bmatrix} \hat{X}$

endif

8. The rationale for the approach of Fact 6 and Algorithm 6 is the following: The computations of $\hat{\mathbf{v}}$ and $\hat{\mathbf{x}}_i$ involve only subtractions of exact quantities, so there is no cancellation. Thus, all entries of each $\hat{\mathbf{x}}_i$ are computed with high relative accuracy so $\hat{\mathbf{x}}_i$ are mutually orthonormal to working

precision. Also, the transition from the matrix $\hat{\Lambda} + e_k \mathbf{v} \mathbf{v}^T$ to the matrix $\hat{\Lambda} + e_k \hat{\mathbf{v}} \hat{\mathbf{v}}^T$ induces only perturbations that are bounded by $\epsilon \|T\|$. Thus, the EVD computed by Algorithm 6 satisfies the error bounds given in Fact 15 in section 42.1, producing at the same time numerically orthogonal eigenvectors. For details see [Dem97, pp. 224–226] and [GE95].

9. Although Algorithm 6 requires $O(n^3)$ operations (this is due to the computation of U in the last line), it is in practice usually faster than Algorithm 4 from Fact 2 in section 42.3. This is due to deflations which are performed when solving the secular equation from Fact 3, resulting in matrix $\hat{X}$ having many zeros.
10. The operation count of Algorithm 6 can be reduced to $O(n^2 \log n)$ if the Fast Multipole Method, originally used in particle simulation, is used for solving secular equation from Fact 3 and for multiplying $\hat{U} \hat{X}$ in the last line of Algorithm 6. For details see [Dem97, pp. 227–228] and [GE95].

Examples:

1. Let T be the matrix from Example 1 in section 42.2 pre- and postmultiplied by the matrix $D = \text{diag}(1, -1, -1, 1)$:

$$T = \begin{bmatrix} 4.5013 & 3.0194 & 0 & 0 \\ 3.0194 & -0.3692 & 1.2804 & 0 \\ 0 & 1.2804 & 0.5243 & 0.9303 \\ 0 & 0 & 0.9303 & 2.6774 \end{bmatrix}.$$

The EVDs of the matrices $\hat{T}_1$ and $\hat{T}_2$ from Equation 42.6 in Fact 1 are

$$\begin{aligned} \hat{T}_1 &= \begin{bmatrix} 4.5013 & 3.0194 \\ 3.0194 & -1.6496 \end{bmatrix}, \quad \hat{U}_1 = \begin{bmatrix} 0.3784 & -0.9256 \\ -0.9256 & -0.3784 \end{bmatrix}, \quad \hat{\Lambda}_1 = \begin{bmatrix} -2.8841 & 0 \\ 0 & 5.7358 \end{bmatrix}, \\ \hat{T}_2 &= \begin{bmatrix} -0.7561 & 0.9303 \\ 0.9303 & 2.6774 \end{bmatrix}, \quad \hat{U}_2 = \begin{bmatrix} -0.9693 & -0.2458 \\ 0.2458 & -0.9693 \end{bmatrix}, \quad \hat{\Lambda}_2 = \begin{bmatrix} -0.9920 & 0 \\ 0 & 2.9132 \end{bmatrix}, \end{aligned}$$

so, in Equation 42.7 in Fact 1, we have

$$\begin{aligned} \hat{\Lambda} &= \text{diag}(-2.8841, 5.7358, -0.9920, 2.9132), \\ \mathbf{v} &= [-0.9256 \quad -0.3784 \quad -0.9693 \quad -0.2458]^T. \end{aligned}$$

2. Algorithm 6 is implemented in the LAPACK subroutine DSTEDC. This routine can compute just the eigenvalues or both, eigenvalues and eigenvectors. The routine requires workspace of approximately n^2 elements. The timings are given in Section 42.9.

42.5 Bisection and Inverse Iteration

The bisection method is convenient if only part of the spectrum is needed. If the eigenvectors are needed, as well, they can be efficiently computed by the inverse iteration method (see Facts 7 and 8 in Section 42.1).

Facts:

The following facts can be found in [Dem97, pp. 228–213] and [Par80, pp. 65–75].

A is a real symmetric $n \times n$ matrix and T is a real symmetric tridiagonal $n \times n$ matrix.

1. (Sylvester's theorem) For a real nonsingular matrix X , the matrices A and $X^T A X$ have the same inertia. (See also Section 8.3.)

2. Let $\alpha, \beta \in \mathbb{R}$ with $\alpha < \beta$. The number of eigenvalues of A in the interval $[\alpha, \beta)$ is equal to $\nu(A - \beta I) - \nu(A - \alpha I)$. By systematically choosing the intervals $[\alpha, \beta)$, the bisection method pinpoints each eigenvalue of A to any desired accuracy.
3. In the factorization $T - \mu I = LDL^T$, where $D = \text{diag}(d_1, \dots, d_n)$ and L is the unit lower bidiagonal matrix, the elements of D are computed by the recursion

$$d_1 = t_{11} - \mu, \quad d_i = (t_{ii} - \mu) - t_{i,i-1}^2/d_{i-1}, \quad i = 2, \dots, n,$$

and the subdiagonal elements of L are given by $l_{i+1,i} = t_{i+1,i}/d_i$. By Fact 1 the matrices T and D have the same inertia, thus the above recursion enables an efficient implementation of the bisection method for T .

4. The factorization from Fact 3 is essentially Gaussian elimination without pivoting. Nevertheless, if $d_i \neq 0$ for all i , the above recursion is very stable (see [Dem97, Lemma 5.4] for details).
5. Even when $d_{i-1} = 0$ for some i , if the IEEE arithmetic is used, the computation will continue and the inertia will be computed correctly. Namely, in that case, we would have $d_i = -\infty$, $l_{i+1,i} = 0$, and $d_{i+1} = t_{i+1,i+1} - \mu$. For details see [Dem97, pp. 230–231] and the references therein.
6. Computing one eigenvalue of T by using the recursion from Fact 3 and bisection requires $O(n)$ operations. For a computed eigenvalue the corresponding eigenvector is computed by inverse iteration given by Equation 42.2. The convergence is very fast (Fact 7 in Section 42.1), so the cost of computing each eigenvector is also $O(n)$ operations. Therefore, the overall cost for computing all eigenvalues and eigenvectors is $O(n^2)$ operations.
7. Both, bisection and inverse iteration are highly parallel since each eigenvalue and eigenvector can be computed independently.
8. If some of the eigenvalues are too close, the corresponding eigenvectors computed by inverse iteration may not be sufficiently orthogonal. In this case, it is necessary to orthogonalize these eigenvectors (for example, by the modified Gram–Schmidt procedure). If the number of close eigenvalues is too large, the overall operation count can increase to $O(n^3)$.
9. The EVD computed by bisection and inverse iteration satisfies the error bounds from Fact 15 in Section 42.1.

Examples:

1. The bisection method for tridiagonal matrices is implemented in the LAPACK subroutine DSTEBZ. This routine can compute all eigenvalues in a given interval or the eigenvalues from λ_l to λ_k , where $l < k$, and the eigenvalues are ordered from smallest to largest. Inverse iteration (with reorthogonalization) is implemented in the LAPACK subroutine DSTEIN. The timings for computing half of the largest eigenvalues and the corresponding eigenvectors are given in Section 42.9.

42.6 Multiple Relatively Robust Representations

The computation of the tridiagonal EVD which satisfies the error bounds of Fact 15 in section 42.1 such that the eigenvectors are orthogonal to working precision, all in $O(n^2)$ operations, has been the “holy grail” of numerical linear algebra for a long time. The method of Multiple Relatively Robust Representations (MRRR) does the job, except in some exceptional cases. The key idea is to implement inverse iteration more carefully. The practical algorithm is quite elaborate and only main ideas are described here.

Facts:

The following facts can be found in [Dhi97], [DP04], and [DPV04].

$T = [t_{ij}]$ denotes a real symmetric tridiagonal matrix of order n . D , D_+ , and D_- are diagonal matrices with the i -th diagonal entry denoted by d_i , $D_+(i)$, and $D_-(i)$, respectively. L and L_+ are unit lower bidiagonal matrices and U_- is a unit upper bidiagonal matrix, where we denote $(L)_{i+1,i}$ by l_i , $(L_+)_{i+1,i}$ by $L_+(i)$, and $(U_-)_{i,i+1}$ by $U_-(i)$.

1. Instead of working with the given T , the MRRR method works with the factorization $T = LDL^T$ (computed, for example, as in Fact 3 in Section 42.5 with $\mu = 0$). If T is positive definite, then all eigenvalues of LDL^T are determined to high relative accuracy in the sense that small relative changes in the elements of L and D cause only small relative changes in the eigenvalues. If T is indefinite, then the tiny eigenvalues of LDL^T are determined to high relative accuracy in the same sense. The bisection method based on Algorithms 7a and 7b computes the well determined eigenvalues of LDL^T to high relative accuracy, that is, the computed eigenvalue $\hat{\lambda}$ satisfies $|\lambda - \hat{\lambda}| = O(n\epsilon|\hat{\lambda}|)$.
2. The MRRR method is based on the following three algorithms:

Algorithm 7a: Differential stationary qd transformInput: factors L and D of T and the computed eigenvalue $\hat{\lambda}$ Output: matrices D_+ and L_+ such that $LDL^T - \hat{\lambda}I = L_+D_+L_+^T$ and vector $\mathbf{s}$

$$s_1 = -\hat{\lambda}$$

for $i = 1 : n - 1$

$$D_+(i) = s_i + d_i$$

$$L_+(i) = (d_i l_i) / D_+(i)$$

$$s_{i+1} = L_+(i) l_i s_i - \hat{\lambda}$$

endfor

$$D_+(n) = s_n + d_n$$

Algorithm 7b: Differential progressive qd transformInput: factors L and D of T and the computed eigenvalue $\hat{\lambda}$ Output: matrices D_- and U_- such that $LDL^T - \hat{\lambda}I = U_-D_-U_-^T$ and vector $\mathbf{p}$

$$p_n = d_n - \hat{\lambda}$$

for $i = n - 1 : -1 : 1$

$$D_-(i + 1) = d_i l_i^2 + p_{i+1}$$

$$t = d_i / D_-(i + 1)$$

$$U_-(i) = l_i t$$

$$p_i = p_{i+1} t - \hat{\lambda}$$

endfor

$$D_-(1) = p_1$$

Algorithm 7c: Eigenvector computationInput: output of Algorithms 7a and 7b and the computed eigenvalue $\hat{\lambda}$ Output: index r and the eigenvector $\mathbf{u}$ such that $LDL^T \mathbf{u} = \hat{\lambda} \mathbf{u}$.**for** $i = 1 : n - 1$

$$\gamma_i = s_i + \frac{d_i}{D_-(i+1)} p_{i+1}$$

endfor

$$\gamma_n = s_n + p_n + \hat{\lambda}$$

find r such that $|\gamma_r| = \min_i |\gamma_i|$

$$u_r = 1$$

for $i = r - 1 : -1 : 1$

$$u_i = -L_+(i) u_{i+1}$$

endfor**for** $i = r : n - 1$

$$u_{i+1} = -U_-(i) u_i$$

endfor

$$\mathbf{u} = \mathbf{u} / \|\mathbf{u}\|_2$$

3. Algorithm 7a is accurate in the sense that small relative perturbations (of the order of few ϵ) in the elements l_i , d_i , and the computed elements $L_+(i)$ and $D_+(i)$ make $L D L^T - \hat{\lambda} I = L_+ D_+ L_+^T$ an exact equality. Similarly, Algorithm 7b is accurate in the sense that small relative perturbations in the elements l_i , d_i , and the computed elements $U_-(i)$ and $D_-(i)$ make $L D L^T - \hat{\lambda} I = U_- D_- U_-^T$ an exact equality.
4. The idea behind the Algorithm 7c is the following: Index r is the index of the column of the matrix $(L D L^T - \hat{\lambda} I)^{-1}$ with the largest norm. Since the matrix $L D L^T - \hat{\lambda} I$ is nearly singular, the eigenvector is computed in just one step of inverse iteration given by Equation 42.2 starting from the vector $\gamma_r \mathbf{e}_r$. Further, $L D L^T - \hat{\lambda} I = N \Delta N^T$, where $N \Delta N^T$ is the so-called twisted factorization obtained from L_+ , D_+ , U_- , and D_- :

$$\Delta = \text{diag}(D_+(1), \dots, D_+(r-1), \gamma_r, D_-(r+1), \dots, D_-(n)),$$

$$N_{ii} = 1, \quad N_{i+1,i} = L_+(i), \quad i = 1, \dots, r-1, \quad N_{i,i+1} = U_-(i), \quad i = r, \dots, n-1.$$

Since $\Delta \mathbf{e}_r = \gamma_r \mathbf{e}_r$ and $N \mathbf{e}_r = \mathbf{e}_r$, solving $N \Delta N^T \mathbf{u} = \gamma_r \mathbf{e}_r$ is equivalent to solving $N^T \mathbf{u} = \mathbf{e}_r$, which is exactly what is done by Algorithm 7c.

5. If an eigenvalue λ is well separated from other eigenvalues in the relative sense (the quantity $\min_{\mu \in \sigma(A), \mu \neq \lambda} |\lambda - \mu|/|\lambda|$ is large, say greater than 10^{-3}), then the computed vector $\hat{\mathbf{u}}$ satisfies $\|\sin \Theta(\mathbf{u}, \hat{\mathbf{u}})\|_2 = O(n\epsilon)$. If all eigenvalues are well separated from each other, then the computed EVD satisfies error bounds of Fact 15 in Section 42.1 and the computed eigenvectors are numerically orthogonal, that is, $|\hat{\mathbf{u}}_i^T \hat{\mathbf{u}}_j| = O(n\epsilon)$ for $i \neq j$.
6. If there is a cluster of poorly separated eigenvalues which is itself well separated from the rest of $\sigma(A)$, the MRRR method chooses a shift μ which is near one end of the cluster and computes a new factorization $L D L^T - \mu I = L_+ D_+ L_+^T$. The eigenvalues within the cluster are then recomputed by bisection as in Fact 1 and their corresponding eigenvectors are computed by Algorithms 7a, 7b, and 7c. When properly implemented, this procedure results in the computed EVD, which satisfies the error bounds of Fact 15 in Section 42.1 and the computed eigenvectors are numerically orthogonal.

Examples:

1. The MRRR method is implemented in the LAPACK subroutine DSTEGR. This routine can compute just the eigenvalues, or both eigenvalues and eigenvectors. The timings are given in Section 42.9.

42.7 Jacobi Method

The Jacobi method is the oldest method for EVD computations [Jac846]. The method does not require tridiagonalization. Instead, the method computes a sequence of orthogonally similar matrices which converge to Λ . In each step a simple plane rotation which sets one off-diagonal element to zero is performed.

Definitions:

A is a real symmetric matrix of order n and $A = U \Lambda U^T$ is its EVD.

The **Jacobi method** forms a sequence of matrices,

$$A_0 = A, \quad A_{k+1} = G(i_k, j_k, c, s) A_k G(i_k, j_k, c, s)^T, \quad k = 1, 2, \dots,$$

where $G(i_k, j_k, c, s)$ is the plane rotation matrix defined in Chapter 38.4. The parameters c and s are chosen such that $[A_{k+1}]_{i_k j_k} = [A_{k+1}]_{j_k i_k} = 0$ and are computed as described in Fact 1.

The plane rotation with c and s as above is also called the **Jacobi rotation**.

The **off-norm** of A is defined as $\text{off}(A) = (\sum_i \sum_{j \neq i} a_{ij}^2)^{1/2}$, that is, off-norm is the Frobenius norm of the matrix consisting of all off-diagonal elements of A .

The choice of **pivot elements** $[A_k]_{i_k j_k}$ is called the **pivoting strategy**.

The **optimal pivoting** strategy, originally used by Jacobi, chooses pivoting elements such that $|[A_k]_{i_k j_k}| = \max_{i < j} |[A_k]_{ij}|$.

The **row cyclic** pivoting strategy chooses pivot elements in the systematic row-wise order,

$$(1, 2), (1, 3), \dots, (1, n), (2, 3), (2, 4), \dots, (2, n), (3, 4), \dots, (n-1, n).$$

Similarly, the column-cyclic strategy chooses pivot elements column-wise.

One pass through all matrix elements is called **cycle** or **sweep**.

Facts:

The Facts 1 to 8 can be found in [Wil65, pp. 265–282], [Par80, §9], [GV96, §8.4], and [Dem97, §5.3.5].

1. The Jacobi rotations parameters c and s are computed as follows: If $[A_k]_{i_k j_k} = 0$, then $c = 1$ and $s = 0$, otherwise

$$\tau = \frac{[A_k]_{i_k i_k} - [A_k]_{j_k j_k}}{2[A_k]_{i_k j_k}}, \quad t = \frac{\text{sign}(\tau)}{|\tau| + \sqrt{1 + \tau^2}}, \quad c = \frac{1}{\sqrt{1 + t^2}}, \quad s = c \cdot t.$$

2. After each rotation, the off-norm decreases, that is,

$$\text{off}^2(A_{k+1}) = \text{off}^2(A_k) - 2[A_k]_{i_k j_k}^2.$$

With the appropriate pivoting strategy, the method converges in the sense that

$$\text{off}(A_k) \rightarrow 0, \quad A_k \rightarrow \Lambda, \quad \prod_{k=1}^{\infty} R_{(i_k, j_k)}^T \rightarrow U.$$

3. For the optimal pivoting strategy the square of the pivot element is greater than the average squared element, $[A_k]_{i_k j_k}^2 \geq \text{off}^2(A) \frac{1}{n(n-1)}$. Thus,

$$\text{off}^2(A_{k+1}) \leq \left(1 - \frac{2}{n(n-1)}\right) \text{off}^2(A_k)$$

and the method converges.

4. For the row cyclic and the column cyclic pivoting strategies, the method converges. The convergence is ultimately quadratic in the sense that $\text{off}(A_{k+n(n-1)/2}) \leq \gamma \text{off}^2(A_k)$ for some constant γ , provided $\text{off}(A_k)$ is sufficiently small.
5. We have the following algorithm:

Algorithm 8: Jacobi method with row-cyclic pivoting strategy

Input: real symmetric $n \times n$ matrix A

Output: the eigenvalue matrix Λ and the eigenvector matrix U

$U = I_n$

repeat % one cycle

for $i = 1 : n - 1$

for $j = i + 1 : n$

 compute c and s according to Fact 1

$$\begin{bmatrix} A_{i,1:n} \\ A_{j,1:n} \end{bmatrix} = G(i, j, c, s) \begin{bmatrix} A_{i,1:n} \\ A_{j,1:n} \end{bmatrix}$$

$$\begin{bmatrix} A_{1:n,i} & A_{1:n,j} \end{bmatrix} = \begin{bmatrix} A_{1:n,i} & A_{1:n,j} \end{bmatrix} G(i, j, c, s)^T$$

$$\begin{bmatrix} U_{1:n,i} & U_{1:n,j} \end{bmatrix} = \begin{bmatrix} U_{1:n,i} & U_{1:n,j} \end{bmatrix} G(i, j, c, s)^T$$

endfor

endfor

until $\text{off}(A) \leq \text{tol}$ for some user defined stopping criterion tol

$\Lambda = \text{diag}(A)$

6. Detailed implementation of the Jacobi method can be found in [Rut66] and [WR71].
7. The EVD computed by the Jacobi method satisfies the error bounds from Fact 15 in Section 42.1.
8. The Jacobi method is suitable for parallel computation. There exist convergent parallel strategies which enable simultaneous execution of several rotations.
9. [GV96, p. 429] The Jacobi method is simple, but it is slower than the methods based on tridiagonalization. It is conjectured that standard implementations require $O(n^3 \log n)$ operations. More precisely, each cycle clearly requires $O(n^3)$ operations and it is conjectured that $\log n$ cycles are needed until convergence.
10. [DV92], [DV05] If A is positive definite, the method can be modified such that it reaches the speed of the methods based on tridiagonalization and at the same time computes the eigenvalues with high relative accuracy. (See Chapter 46 for details.)

Examples:

1. Let A be the matrix from Example 1 in section 42.1. After executing two cycles of Algorithm 8, we have

$$A = \begin{bmatrix} 6.0054 & -0.0192 & 0.0031 & 0.0003 \\ -0.0192 & 3.0455 & -0.0005 & -0.0000 \\ 0.0031 & -0.0005 & 0.6024 & -0.0000 \\ 0.0003 & -0.0000 & 0.0000 & -2.3197 \end{bmatrix}.$$

42.8 Lanczos Method

If the matrix A is large and sparse and if only some eigenvalues and their eigenvectors are desired, sparse matrix methods are the methods of choice. For example, the power method can be useful to compute the eigenvalue with the largest modulus. The basic operation in the power method is matrix-vector multiplication, and this can be performed very fast if A is sparse. Moreover, A need not be stored in the computer — the input for the algorithm can be just a program which, given some vector $\mathbf{x}$, computes the product $A\mathbf{x}$. An “improved” version of the power method, which efficiently computes several eigenvalues (either largest in modulus or near some target value μ) and the corresponding eigenvectors, is the Lanczos method.

Definitions:

A is a real symmetric matrix of order n .

Given a nonzero vector $\mathbf{x}$ and an index $k < n$, the **Krylov matrix** is defined as $K_k = [\mathbf{x} \quad A\mathbf{x} \quad A^2\mathbf{x} \quad \cdots \quad A^{k-1}\mathbf{x}]$.

Facts:

The following facts can be found in [Par80, §13], [GV96, §9], [Dem97, §7], and [Ste01, §5.3].

1. The Lanczos method is based on the following observation. If $K_k = XR$ is the QR factorization of the matrix K_k (see Sections 5.5 and 38.4), then the $k \times k$ matrix $T = X^TAX$ is tridiagonal. The matrices X and T can be computed by using only matrix-vector products in just $O(kn)$ operations. Let $T = Q\Lambda Q^T$ be the EVD of T (computed by any of the methods from Sections 42.3 to 42.6). Then λ_i approximate well some of the largest and smallest eigenvalues of A . The columns of the matrix $U = XQ$ approximate the corresponding eigenvectors of A . We have the following algorithm:

Algorithm 9: Lanczos methodInput: real symmetric $n \times n$ matrix A , unit vector $\mathbf{x}$ and index $k < n$ Output: matrices Λ and U $X_{:,1} = \mathbf{x}$ **for** $i = 1 : k$ $\mathbf{z} = A X_{:,i}$ $t_{ii} = X_{:,i}^T \mathbf{z}$ **if** $i = 1$, **then** $\mathbf{z} = \mathbf{z} - t_{ii} X_{:,i}$ **else** $\mathbf{z} = \mathbf{z} - t_{ii} X_{:,i} - t_{i,i-1} X_{:,i-1}$ **endif** $\mu = \|\mathbf{z}\|_2$ **if** $\mu = 0$, **then**

stop

else $t_{i+1,i} = \mu$ $t_{i,i+1} = \mu$ $X_{:,i+1} = \mathbf{z}/\mu$ **endif****endfor**compute the EVD of the tridiagonal matrix, $T(1:k, 1:k) = Q \Lambda Q^T$ $U = XQ$

2. As j increases, the largest (smallest) eigenvalues of the matrix $T_{1:j,1:j}$ converge towards some of the largest (smallest) eigenvalues of A (due to the Cauchy interlace property). The algorithm can be redesigned to compute only largest or smallest eigenvalues. Also, by using shift and invert strategy, the method can be used to compute eigenvalues near some specified value. In order to obtain better approximations, k should be greater than the number of required eigenvalues. On the other side, in order to obtain better accuracy and efficacy, k should be as small as possible (see Facts 3 and 4 below).
3. The eigenvalues of A are approximated from the matrix $T_{1:k,1:k}$, thus, the last element $v = t_{k+1,k}$ is not needed. However, this element provides key information about accuracy at no extra computational cost. The exact values of residuals are as follows: $\|AU - U\Lambda\|_2 = v$ and, in particular, $\|AU_{:,i} - \lambda_i U_{:,i}\|_2 = v|q_{ki}|$, $i = 1, \dots, k$. Further, there are k eigenvalues $\tilde{\lambda}_1, \dots, \tilde{\lambda}_k$ of A such that $|\lambda_i - \tilde{\lambda}_i| \leq v$. For the corresponding eigenvectors, we have $\sin 2\Theta(\mathbf{u}_i, \tilde{\mathbf{u}}_i) \leq 2v / \min_{j \neq i} |\lambda_i - \tilde{\lambda}_j|$. In practical implementations of Algorithm 9, v is usually used to determine the index k .
4. Although theoretically very elegant, the Lanczos method has inherent numerical instability in the floating point arithmetic, and so it must be implemented carefully (see, e.g., [LSY98]). Since the Krylov vectors are, in fact, generated by the power method, they converge towards an eigenvector of A . Thus, as k increases, the Krylov vectors become more and more parallel. As a consequence, the recursion in Algorithm 9, which computes the orthogonal bases X for the subspace range K_k , becomes numerically unstable and the computed columns of X cease to be sufficiently orthogonal. This affects both the convergence and the accuracy of the algorithm. For example, it can happen that T has several eigenvalues which converge towards some simple eigenvalue of A (these are the so called ghost eigenvalues).

The loss of orthogonality is dealt with by using the full reorthogonalization procedure. In each step, the new $\mathbf{z}$ is orthogonalized against all previous columns of X . In Algorithm 9, the formula $\mathbf{z} = \mathbf{z} - t_{ii} X_{:,i} - t_{i,i-1} X_{:,i-1}$ is replaced by $\mathbf{z} = \mathbf{z} - \sum_{j=1}^{i-1} (\mathbf{z}^T X(:, j)) X(:, j)$. To obtain better orthogonality, the latter formula is usually executed twice.

The full reorthogonalization raises the operation count to $O(k^2n)$. The selective reorthogonalization is the procedure in which the current z is orthogonalized against some selected columns of X . This is the way to attain sufficient numerical stability and not increase the operation count too much. The details of selective reorthogonalization procedures are very subtle and can be found in the references. (See also Chapter 44.)

5. The Lanczos method is usually used for sparse matrices. Sparse matrix A is stored in the sparse format in which only values and indices of nonzero elements are stored. The number of operations required to multiply some vector by A is also proportional to the number of nonzero elements. (See also Chapter 43.)

Examples:

1. Let A be the matrix from Example 1 in section 42.1 and let $x = [1/2 \ 1/2 \ 1/2 \ 1/2]^T$. For $k = 2$, the output of Algorithm 9 is

$$\Lambda = \begin{bmatrix} -2.0062 & & \\ & 5.7626 & \end{bmatrix}, \quad U = \begin{bmatrix} -0.4032 & -0.8804 \\ 0.4842 & -0.2749 \\ 0.3563 & -0.3622 \\ 0.6899 & -0.1345 \end{bmatrix},$$

with $\nu = 1.4965$ (c.f. Fact 3). For $k = 3$, the output is

$$\Lambda = \begin{bmatrix} -2.3107 & 0 & 0 \\ 0 & 2.8641 & 0 \\ 0 & 0 & 5.9988 \end{bmatrix}, \quad U = \begin{bmatrix} 0.3829 & -0.0244 & 0.8982 \\ -0.2739 & -0.9274 & 0.0312 \\ -0.3535 & -0.1176 & 0.3524 \\ -0.8084 & 0.3541 & 0.2607 \end{bmatrix},$$

with $\nu = 0.6878$.

2. The Lanczos method is implemented in the ARPACK routine DSDRV*, where * denotes the computation mode [LSY98, App. A]. The routines from ARPACK are implemented in the MATLAB command `eigs`. Generation of a sparse symmetric $10,000 \times 10,000$ matrix with 10% nonzero elements with the MATLAB command `A=sprandsym(10000,0.1)` takes 15 seconds on a processor described in Fact 1 in section 42.9. The computation of 100 largest eigenvalues and the corresponding eigenvectors with `[U,Lambda]=eigs(A,100,'LM',opts)` takes 140 seconds. Here, index $k = 200$ is automatically chosen by the algorithm. (See also Chapter 76.)

42.9 Comparison of Methods

In this section, we give timings for the LAPACK implementations of the methods described in Sections 42.2 to 42.6. The timing for the Lanczos method is given in Example 2 in Section 42.8.

Definitions:

A measure of processor's **efficacy** or **speed** is the number of floating-point operations per second (flops).

Facts:

A is an $n \times n$ real symmetric matrix and $A = U\Lambda U^T$ is its EVD. T is a tridiagonal $n \times n$ real symmetric matrix and $T = Q\Lambda Q^T$ is its EVD. $T = X^T A X$ is the reduction of A to a tridiagonal from Section 42.2.

1. Our tests were performed on the Intel Xeon processor running at 2.8 MHz with 2 Mbytes of cache memory. This processor performs up to 5 Gflops (5 billion operations per second). The peak performance is attained for the matrix multiplication with the BLAS 3 subroutine DGEMM.

TABLE 42.1 Execution times(s) for LAPACK routines for various matrix dimensions n .

Routine	Input	Output	Example	$n = 500$	$n = 1000$	$n = 2000$
DSYTRD	A	T	2.3	0.10	0.78	5.5
DSYTRD/DORGTR		T, X		0.17	1.09	8.6
DSTEQR	T	Λ	3.2	0.03	0.11	0.44
		Λ, Q		0.32	2.23	15.41
DSYEV	A	Λ	3.4	0.12	0.85	5.63
		Λ, U		0.46	3.13	22.30
DSTEDC	T	Λ	4.2	0.02	0.08	0.28
		Λ, Q		0.05	0.12	0.36
DSTEBZ	T	Λ	5.1	0.21	0.81	3.15
DSTEIN		Q		0.04	0.17	0.72
DSTEGR	T	Λ	6.1	0.07	0.25	0.87
		Λ, Q		0.09	0.35	1.29

- Our test programs were compiled with the Intel `ifort` FORTRAN compiler (version 9.0) and linked with the Intel Math Kernel Library (version 8.0.2).
- Timings for the methods are given in Table 42.1. The execution times for DSTEBZ (bisection) and DSTEIN (inverse iteration) are for computing one half of the eigenvalues (the largest ones) and the corresponding eigenvectors, respectively.
- The performance attained for practical algorithms is lower than the peak performance from Fact 1. For example, by combining Facts 6 and 7 in Section 42.2 with Table 42.1, we see that the tridiagonalization routines DSYTRD and DORGTR attain the speed of 2 Gflops.
- The computation times for the implicitly shifted QR routine, DSTEQR, grow with n^2 when only eigenvalues are computed, and with n^3 when eigenvalues and eigenvectors are computed, as predicted in Facts 5 and 6 in Section 42.3.
- The execution times for DSYEV are approximately equal to the sums of the timings for DSYTRD (tridiagonalization), DORGTR (computing X), and DSTEQR with the eigenvector option (computing the EVD of T).
- The divide and conquer method, implemented in DSTEDC, is the fastest method for computing the EVD of a tridiagonal matrix.
- DSTEBZ and DSTEIN (bisection and inverse iteration) are faster, especially for larger dimensions, than DSTEQR (tridiagonal QR iteration), but slower than DSTEDC (divide and conquer) and DSTEGR (multiple relatively robust representations).
- Another algorithm to compute the EVD of T is to use DSTEQR to compute only the eigenvalues and then use DSTEIN (inverse iteration) to compute the eigenvectors. This is usually considerably faster than computing both, eigenvalues and eigenvectors, by DSTEQR.
- The executions times for DSTEGR are truly proportional to $O(n^2)$.
- The new LAPACK release, in which some of the above mentioned routines are improved with respect to speed and/or accuracy, is announced for the second half of 2006.

References

- [ABB99] E. Anderson, Z. Bai, and C. Bischof, *LAPACK Users' Guide*, 3rd ed., SIAM, Philadelphia, 1999.
- [Cup81] J.J.M. Cuppen, A divide and conquer method for the symmetric tridiagonal eigenproblem, *Numer. Math.*, 36:177–195, 1981.
- [Dem97] J.W. Demmel, *Applied Numerical Linear Algebra*, SIAM, Philadelphia, 1997.
- [DV92] J.W. Demmel and K. Veselić, Jacobi's method is more accurate than QR, *SIAM J. Matrix Anal. Appl.*, 13:1204–1245, 1992.

- [Dhi97] I.S. Dhillon, *A New $O(N^2)$ Algorithm for the Symmetric Tridiagonal Eigenvalue/Eigenvector Problem*, Ph.D. thesis, University of California, Berkeley, 1997.
- [DP04] I.S. Dhillon and B.N. Parlett, Orthogonal eigenvectors and relative gaps, *SIAM J. Matrix Anal. Appl.*, 25:858–899, 2004.
- [DPV04] I.S. Dhillon, B.N. Parlett, and C. Vömel, “The Design and Implementation of the MRRR Algorithm,” Tech. Report UCB/CSD-04-1346, University of California, Berkeley, 2004.
- [DHS89] J.J. Dongarra, S.J. Hammarling, and D.C. Sorensen, Block reduction of matrices to condensed forms for eigenvalue computations, *J. Comp. Appl. Math.*, 27:215–227, 1989.
- [DV05] Z. Drmač and K. Veselić, New fast and accurate Jacobi SVD algorithm: I, Technical report, University of Zagreb, 2005, also LAPACK Working Note #169.
- [GV96] G.H. Golub and C.F. Van Loan, *Matrix Computations*, 3rd ed., The John Hopkins University Press, Baltimore, MD, 1996.
- [GE95] M. Gu and S.C. Eisenstat, A divide-and-conquer algorithm for the symmetric tridiagonal eigenproblem, *SIAM J. Matrix Anal. Appl.*, 16:79–92, 1995.
- [Jac846] C.G.J. Jacobi, Über ein leichtes Verfahren die in der Theorie der Säcularstörungen vorkommenden Gleichungen numerisch aufzulösen, *Crelles Journal für Reine und Angew. Math.*, 30:51–95, 1846.
- [LSY98] R.B. Lehoucq, D.C. Sorensen, and C. Yang, *ARPACK Users’ Guide: Solution of Large-Scale Eigenvalue Problems with Implicitly Restarted Arnoldi Methods*, SIAM, Philadelphia, 1998.
- [Par80] B.N. Parlett, *The Symmetric Eigenvalue Problem*, Prentice-Hall, Upper Saddle River, NJ, 1980.
- [Rut66] H. Rutishauser, The Jacobi method for real symmetric matrices, *Numerische Mathematik*, 9:1–10, 1966.
- [Ste01] G.W. Stewart, *Matrix Algorithms, Vol. II: Eigensystems*, SIAM, Philadelphia, 2001.
- [TB97] L.N. Trefethen and D. Bau, III, *Numerical Linear Algebra*, SIAM, Philadelphia, 1997.
- [Wil65] J.H. Wilkinson, *The Algebraic Eigenvalue Problem*, Clarendon Press, Oxford, U.K., 1965.
- [WR71] J.H. Wilkinson and C. Reinsch, *Handbook for Automatic Computation, Vol. II, Linear Algebra*, Springer, New York, 1971.

43

Unsymmetric Matrix Eigenvalue Techniques

David S. Watkins
Washington State University

43.1	The Generalized Eigenvalue Problem	43-1
43.2	Dense Matrix Techniques	43-3
43.3	Sparse Matrix Techniques	43-9
	References	43-11

The definitions and basic properties of eigenvalues and eigenvectors are given in Section 4.3. A natural generalization is presented here in Section 43.1. Algorithms for computation of eigenvalues, eigenvectors, and their generalizations will be discussed in Sections 43.2 and 43.3. Although the characteristic equation is important in theory, it plays no role in practical eigenvalue computations.

If a large fraction of a matrix's entries are zeros, the matrix is called **sparse**. A matrix that is not sparse is called **dense**. Dense matrix techniques are methods that store the matrix in the conventional way, as an array, and operate on the array elements. Any matrix that is not too big to fit into a computer's main memory can be handled by dense matrix techniques, regardless of whether the matrix is dense or not. However, since the time to compute the eigenvalues of an $n \times n$ matrix by dense matrix techniques is proportional to n^3 , the user may have to wait awhile for the results if n is very large. Dense matrix techniques do not exploit the zeros in a matrix and tend to destroy them. With modern computers, dense matrix techniques can be applied to matrices of dimension up to 1000 or more. If a matrix is very large and sparse, and only a portion of the spectrum is needed, sparse matrix techniques (Section 43.3) are preferred.

The usual approach is to preprocess the matrix into Hessenberg form and then to effect a similarity transformation to triangular form: $T = S^{-1}AS$ by an iterative method. This yields the eigenvalues of A as the main-diagonal entries of T . For $k = 1, \dots, n-1$, the first k columns of S span an invariant subspace. The eigenvectors of an upper-triangular matrix are easily computed by back substitution, and the eigenvectors of A can be deduced from the eigenvectors of T [GV96, § 7.6], [Wat02, § 5.8]. If a matrix A is very large and sparse, only a partial similarity transformation is possible because a complete similarity transformation would require too much memory and take too long to compute.

43.1 The Generalized Eigenvalue Problem

Many matrix eigenvalue problems are most naturally viewed as generalized eigenvalue problems.

Definitions:

Given $A \in \mathbb{C}^{n \times n}$ and $B \in \mathbb{C}^{n \times n}$, the nonzero vector $\mathbf{v} \in \mathbb{C}^n$ is called an **eigenvector** of the pair (A, B) if there are scalars $\mu, \nu \in \mathbb{C}$, not both zero, such that

$$\nu A\mathbf{v} = \mu B\mathbf{v}.$$

Then, the scalar $\lambda = \mu/\nu$ is called the **eigenvalue** of (A, B) associated with the eigenvector $\mathbf{v}$. If $\nu = 0$, then the eigenvalue is ∞ by convention.

The expression $A - xB$, with indeterminate x , is called a **matrix pencil**. Whether we refer to the pencil $A - xB$ or the pair (A, B) , we are speaking of the same object. The pencil (or the pair (A, B)) is called **singular** if $A - \lambda B$ is singular for all $\lambda \in \mathbb{C}$. The pencil is **regular** if there exists a $\lambda \in \mathbb{C}$ such that $A - \lambda B$ is nonsingular. We will restrict our attention to regular pencils.

The **characteristic polynomial** of the pencil $A - xB$ is $\det(xB - A)$, and the **characteristic equation** is $\det(xB - A) = 0$.

Two pairs (A, B) and (C, D) are **strictly equivalent** if there exist nonsingular matrices S_1 and S_2 such that $C - \lambda D = S_1(A - \lambda B)S_2$ for all $\lambda \in \mathbb{C}$. If S_1 and S_2 can be taken to be unitary, then the pairs are **strictly unitarily equivalent**.

A pair (A, B) is called **upper triangular** if both A and B are upper triangular.

Facts:

The following facts are discussed in [GV96, § 7.7] and [Wat02, § 6.7].

1. When $B = I$, the generalized eigenvalue problem for the pair (A, B) reduces to the standard eigenvalue problem for the matrix A .
2. λ is an eigenvalue of (A, B) if and only if $A - \lambda B$ is singular.
3. λ is an eigenvalue of (A, B) if and only if $\ker(\lambda B - A) \neq \{0\}$.
4. The eigenvalues of (A, B) are exactly the solutions of the characteristic equation $\det(xB - A) = 0$.
5. The characteristic polynomial $\det(xB - A)$ is a polynomial in x of degree $\leq n$.
6. The pair (A, B) (or the pencil $A - xB$) is singular if and only if $\det(\lambda B - A) = 0$ for all λ , if and only if the characteristic polynomial $\det(xB - A)$ is equal to zero.
7. If the pair (A, B) is regular, then $\det(xB - A)$ is a nonzero polynomial of degree $k \leq n$. (A, B) has k finite eigenvalues.
8. The degree of $\det(xB - A)$ is exactly n if and only if B is nonsingular.
9. If B is nonsingular, then the eigenvalues of (A, B) are exactly the eigenvalues of the matrices AB^{-1} and $B^{-1}A$.
10. If $\lambda \neq 0$, then λ is an eigenvalue of (A, B) if and only if λ^{-1} is an eigenvalue of (B, A) .
11. Zero is an eigenvalue of (A, B) if and only if A is a singular matrix.
12. Infinity is an eigenvalue of (A, B) if and only if B is a singular matrix.
13. Two pairs that are strictly equivalent have the same eigenvalues.
14. If $C - \lambda D = S_1(A - \lambda B)S_2$, then $\mathbf{v}$ is an eigenvector of (A, B) if and only if $S_2^{-1}\mathbf{v}$ is an eigenvector of (C, D) .
15. (Schur's Theorem) Every $A \in \mathbb{C}^{n \times n}$ is unitarily similar to an upper triangular matrix S .
16. (Generalized Schur Theorem) Every pair (A, B) is strictly unitarily equivalent to an upper triangular pair (S, T) .
17. The characteristic polynomial of an upper triangular pair (S, T) is $\prod_{k=1}^n (\lambda t_{kk} - s_{kk})$. The eigenvalues of (S, T) are $\lambda_k = s_{kk}/t_{kk}$, $k = 1, \dots, n$. If $t_{kk} = 0$ and $s_{kk} \neq 0$, then $\lambda_k = \infty$. If $t_{kk} = 0$ and $s_{kk} = 0$ for some k , the pair (S, T) is singular.

Examples:

1. Let $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$. Then the characteristic polynomial of the pair (A, B) is $x^2 + x - 2 = 0$, and the eigenvalues are 1 and -2 .

2. Since the pencil

$$\begin{bmatrix} 2 & 5 \\ 0 & 7 \end{bmatrix} - x \begin{bmatrix} 5 & 1 \\ 0 & 3 \end{bmatrix}$$

is upper triangular, its characteristic polynomial is $(5x - 2)(3x - 7)$, and its eigenvalues are $2/5$ and $7/3$.

3. The pencil

$$\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} - x \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

has characteristic equation $x = 0$. It is a regular pencil with eigenvalues 0 and ∞ .

43.2 Dense Matrix Techniques

The steps that are usually followed for solving the unsymmetric eigenvalue problem are preprocessing, eigenvalue computation with the *QR* Algorithm, and eigenvector computation.

The most widely used public domain software for this problem is from LAPACK [ABB99], and Chapter 75. Versions in FORTRAN and C are available. The most popular proprietary software is MATLAB, which uses computational routines from LAPACK. Several of LAPACK's computational routines will be mentioned in this section. LAPACK also has a number of driver routines that call the computational routines to perform the most common tasks, thereby making the user's job easier. A very easy way to use LAPACK routines is to use MATLAB.

This section presents algorithms for the reader's edification. However, the reader is strongly advised to use well-tested software written by experts whenever possible, rather than writing his or her own code. The actual software is very complex and addresses details that cannot be discussed here.

Definitions:

A matrix $A \in \mathbb{C}^{n \times n}$ is called **upper Hessenberg** if $a_{ij} = 0$ whenever $i > j + 1$. This means that every entry below the first subdiagonal of A is zero. An upper Hessenberg matrix is called **unreduced upper Hessenberg** if $a_{j+1,j} \neq 0$ for $j = 1, \dots, n - 1$.

Facts:

The following facts are proved in [Dem97], [GV96], [Kre05], or [Wat02].

1. Preprocessing is a two step process involving balancing the matrix and transforming by unitary similarity to upper Hessenberg form.
2. The first step, which is optional, is to balance the matrix. The balancing operation begins by performing a permutation similarity transformation that exposes any obvious eigenvalues. The remaining submatrix is irreducible. It then performs a diagonal similarity transformation $D^{-1}AD$ that attempts to make the norms of the i th row and i th column as nearly equal as possible, $i = 1, \dots, n$. This has the effect of reducing the overall norm of the matrix and in diminishing the effects of roundoff errors [Osb60]. The scaling factors in D are taken to be powers of the base of floating point arithmetic (usually 2). No roundoff errors are caused by this transformation.
3. All modern balancing routines, including the code GEBAL in LAPACK, are derived from the code in Parlett and Reinsch [PR69]. See also [Kre05].

Algorithm 1: Balancing an Irreducible Matrix. An irreducible matrix $A \in \mathbb{C}^{n \times n}$ is input. On output, A has been overwritten by $D^{-1}AD$, where D is diagonal.

$b \leftarrow$ base of floating point arithmetic (usually 2)

$D \leftarrow I_n$

done $\leftarrow 0$

while done = 0

 done $\leftarrow 1$

 for $j = 1 : n$

$c \leftarrow \sum_{i \neq j} |a_{ij}|, \quad r \leftarrow \sum_{k \neq j} |a_{jk}|$

$s \leftarrow c + r, \quad f \leftarrow 1$

 while $b c < r$

$c \leftarrow b c, \quad r \leftarrow r/b, \quad f \leftarrow b f$

 while $b r < c$

$c \leftarrow c/b, \quad r \leftarrow b r, \quad f \leftarrow f/b$

 if $c + r < 0.95 s$

 done $\leftarrow 0, \quad d_{jj} \leftarrow f d_{jj}$

$A_{1:n,j} \leftarrow f A_{1:n,j}, \quad A_{j,1:n} \leftarrow (1/f) A_{j,1:n}$

 end

4. In most cases balancing will have little effect on the outcome of the computation, but sometimes it results in greatly improved accuracy [BDD00, § 7.2].
5. The second preprocessing step is to transform the matrix to upper Hessenberg form. This is accomplished by a sequence of $n - 2$ steps. On the j th step, zeros are introduced into the j th column.
6. For every $\mathbf{x} \in \mathbb{C}^n$ there is a unitary matrix U such that $U\mathbf{x} = \alpha \mathbf{e}_1$, for some scalar $\alpha \in \mathbb{C}$, where $\mathbf{e}_1$ is the vector having a 1 in the first position and zeros elsewhere. U can be chosen to be a rank-one modification of the identity matrix: $U = I + \mathbf{u}\mathbf{v}^*$. (See Section 38.4 for a discussion of Householder and Givens matrices.)

7.

Algorithm 2. Unitary Similarity Transformation to Upper Hessenberg Form. A general matrix $A \in \mathbb{C}^{n \times n}$ is input. On output, A has been overwritten by an upper Hessenberg matrix Q^*AQ . The unitary transforming matrix Q has also been generated.

$Q \leftarrow I_n$

for $j = 1 : n - 2$

 Let $\mathbf{x} = A_{j+1:n,j} \in \mathbb{C}^{n-j}$.

 Build unitary $U \in \mathbb{C}^{(n-j) \times (n-j)}$ such that $U^*\mathbf{x} = \gamma \mathbf{e}_1$.

$A_{j+1:n,j:n} \leftarrow U^* A_{j+1:n,j:n}$

$A_{1:n,j+1:n} \leftarrow A_{1:n,j+1:n} U$

$Q_{1:n,j+1:n} \leftarrow Q_{1:n,j+1:n} U$

end

8. The cost of the reduction to Hessenberg form is proportional to n^3 for large n ; that is, it is $O(n^3)$.
9. For large matrices, efficient cache use can be achieved by processing several columns at a time. This allows the processor(s) to run at much closer to maximum speed. See [GV96, p. 225], [Wat02, p. 210], and the LAPACK code GEHRD [ABB99].

10. Once the matrix is in upper Hessenberg form, if any of the subdiagonal entries $a_{j+1,j}$ is zero, the matrix is block upper triangular with a $j \times j$ block and an $n - j \times n - j$ block, and the eigenvalue problem decouples to two independent problems of smaller size. Thus, we always work with unreduced upper Hessenberg matrices.
11. In practice we set an entry $a_{j+1,j}$ to zero whenever

$$|a_{j+1,j}| < \epsilon(|a_{jj}| + |a_{j+1,j+1}|),$$

where ϵ is the computer's unit roundoff.

12. If $T \in \mathbb{C}^{n \times n}$ is upper triangular and nonsingular, then T^{-1} is upper triangular. If $H \in \mathbb{C}^{n \times n}$ is upper Hessenberg, then TH , HT , and THT^{-1} are upper Hessenberg.
13. The standard method for computing the eigenvalues of a Hessenberg matrix is the QR algorithm, an iterative method that produces a sequence of unitarily similar matrices that converges to upper triangular form.
14. The most basic version of the QR algorithm starts with $A_0 = A$, an unreduced upper Hessenberg matrix, and generates a sequence (A_m) as follows: Given A_{m-1} , a decomposition $A_{m-1} = Q_m R_m$, where Q_m is unitary and R_m is upper triangular, is computed. Then the factors are multiplied back together in reverse order to yield $A_m = R_m Q_m$. Equivalently $A_m = Q_m^* A_{m-1} Q_m$.
15. Upper Hessenberg form is preserved by iterations of the QR algorithm.
16. The QR algorithm can also be applied to non-Hessenberg matrices, but the operations are much more economical in the Hessenberg case.
17. The basic QR algorithm converges slowly, so shifts of origin are used to accelerate convergence:

$$A_{m-1} - \mu_m I = Q_m R_m, \quad R_m Q_m + \mu_m I = Q_m^* A_{m-1} Q_m = A_m,$$

where $\mu_m \in \mathbb{C}$ is a shift chosen to approximate an eigenvalue.

18. Often it is convenient to take several steps at once:

Algorithm 3. Explicit QR iteration of degree k .

Choose k shifts $\mu_1, \dots, \mu_k$.

Let $p(A) = (A - \mu_1 I)(A - \mu_2 I) \cdots (A - \mu_k I)$.

Compute a QR decomposition $p(A) = QR$.

$A \leftarrow Q^* A Q$

19. A QR iteration of degree k is equivalent to k iterations of degree 1 with shifts $\mu_1, \dots, \mu_k$ applied in succession in any order [Wat02]. Upper Hessenberg form is preserved. In practice k is never taken very big; typical values are 1, 2, 4, and 6.
20. One important application of multiple steps is to complex shifts applied to real matrices. Complex arithmetic is avoided by taking $k = 2$ and shifts related by $\mu_2 = \overline{\mu_1}$.
21. The usual choice of k shifts is the set of eigenvalues of the lower right-hand $k \times k$ submatrix of the current iterate. With this choice of shifts at each iteration, the entry $a_{n-k+1,n-k}$ typically converges to zero quadratically [WE91], isolating a $k \times k$ submatrix after only a few iterations. However, convergence is not guaranteed, and failures do occasionally occur. No shifting strategy that guarantees convergence in all cases is known. For discussions of shifting strategies and convergence see [Wat02] or [WE91].
22. After each iteration, all of the subdiagonal entries should be checked to see if any of them can be set to zero. The objective is to break the big problem into many small problems in as few iterations as possible. Once a submatrix of size 1×1 has been isolated, an eigenvalue has been found. The eigenvalues of a 2×2 submatrix can be found by careful use of the quadratic formula. Complex conjugate eigenvalues of real matrices are extracted in pairs.

23. The explicit QR iteration shown above is expensive and never used in practice. Instead the iteration is performed implicitly.

Algorithm 4: Implicit QR iteration of degree k (chasing the bulge).

Choose k shifts $\mu_1, \dots, \mu_k$.

$\mathbf{x} \leftarrow \mathbf{e}_1$ % first column of identity matrix

for $j = 1 : k$

$\mathbf{x} \leftarrow (A - \mu_k I)\mathbf{x}$

end % $\mathbf{x}$ is the first column of $p(A)$.

$\hat{\mathbf{x}} \leftarrow \mathbf{x}_{1:k+1}$ % $\mathbf{x}_{k+2:n} = 0$

Let $U \in \mathbb{C}^{(k+1) \times (k+1)}$ be unitary with $U^* \hat{\mathbf{x}} = \alpha \mathbf{e}_1$

$A_{1:k+1,1:n} \leftarrow U^* A_{1:k+1,1:n}$

$A_{1:n,1:k+1} \leftarrow A_{1:n,1:k+1} U$

Return A to upper Hessenberg form as in Algorithm 2 (Fact 7).

24. The initial transformation in the implicit QR iteration disturbs the upper Hessenberg form of A , making a bulge in the upper left-hand corner. The size of the bulge is equal to k . In the case $k = 2$, the pattern of nonzeros is

$$\begin{bmatrix} * & * & * & * & * & * \\ * & * & * & * & * & * \\ * & * & * & * & * & * \\ * & * & * & * & * & * \\ & & & * & * & * \\ & & & & * & * \end{bmatrix}.$$

The subsequent reduction to Hessenberg form chases the bulge down through the matrix and off the bottom. The equivalence of the explicit and implicit QR iterations is demonstrated in [GV96, § 7.5] and [Wat02, § 5.7]. For this result it is crucial that the matrix is unreduced upper Hessenberg.

25. For a fixed small value of k , the implicit QR iteration requires only $O(n^2)$ work. Typically only a small number of iterations, independent of n , are needed per eigenvalue found; the total number of iterations is $O(n)$. Thus, the implicit QR algorithm is considered to be an $O(n^3)$ process.
26. The main unsymmetric QR routine in LAPACK [ABB99] is HSEQR, a multishift implicit QR algorithm with $k = 6$. For processing small submatrices (50×50 and under), HSEQR calls LAHQR, a multishift QR code with $k = 2$. Future versions of LAPACK will include improved QR routines that save work by doing aggressive early deflation [BBM02b] and make better use of cache by chasing bulges in bunches and aggregating the transforming matrices [BBM02a].
27. If eigenvectors are wanted, the aggregate similarity transformation matrix S , the product of all transformations from start to finish, must be accumulated. $T = S^{-1}AS$, where A is the original matrix and T is the final upper triangular matrix. In the real case, T will not quite be upper triangular. It is **quasi-triangular** with a 2×2 block along the main diagonal for each complex conjugate pair of eigenvalues. This causes complications in the descriptions of the algorithms, but does not cause any practical problems.
28. The eigenvectors of T are computed by back substitution [Wat02, § 5.8]. For each eigenvector $\mathbf{x}$ of T , $S\mathbf{x}$ is an eigenvector of A . The total additional cost of the eigenvector computation is $O(n^3)$. In LAPACK these tasks are performed by the routines HSEQR and TREVC.
29. Invariant subspaces can also be computed. The eigenvalues of A are $\lambda_1 = t_{11}, \dots, \lambda_n = t_{nn}$. If $\lambda_1, \dots, \lambda_k$ are disjoint from $\lambda_{k+1}, \dots, \lambda_n$, then, because T is upper triangular, the first k columns of S span the invariant subspace associated with $\{\lambda_1, \dots, \lambda_k\}$.

30. If an invariant subspace associated with k eigenvalues that are not at the top of T is wanted, then those k eigenvalues must be moved to the top by a sequence of swapping operations. Each operation is a unitary similarity transformation that reverses the positions of two adjacent main-diagonal entries of T . The transformations are applied to S as well. Once the desired eigenvalues have been moved to the top, the first k columns of the transformed S span the desired invariant subspace. For details see [BD93] and [GV96, § 7.6]. In LAPACK these tasks are performed by the routines TREXC and TRSEN.
31. An important difference between the symmetric and unsymmetric eigenvalue problems is that in the unsymmetric case, the eigenvalues can be ill conditioned. That is, a small perturbation in the entries of A can cause a large change in the eigenvalues. Suppose λ is an eigenvalue of A of algebraic multiplicity 1, and let E be a perturbation that is small in the sense that $\|E\|_2 \ll \|A\|_2$. Then $A + E$ has an eigenvalue $\lambda + \delta$ near λ . A condition number for λ is the smallest number κ such that

$$|\delta| \leq \kappa \|E\|_2$$

for all small perturbations E . If $\mathbf{x}$ and $\mathbf{y}$ are eigenvectors of A and A^T , respectively, associated with λ , then [Wat02, § 6.5]

$$\kappa \approx \frac{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2}{|\mathbf{y}^T \mathbf{x}|}.$$

If $\kappa \gg 1$, λ is ill conditioned. If κ is not much bigger than 1, λ is well conditioned.

32. Condition numbers can also be defined for eigenvectors and invariant subspaces [GV96, § 7.2], [Wat02, § 6.5]. Eigenvectors associated with a tight cluster of eigenvalues are always ill conditioned. A more meaningful object is the invariant subspace associated with all of the eigenvalues in the cluster. This space will usually be well conditioned, even though the eigenvectors are ill conditioned. The LAPACK routines TRSNA and TRSEN compute condition numbers for eigenvalues, eigenvectors, and invariant subspaces.
33. The invariant subspace associated with $\{\lambda_1, \dots, \lambda_k\}$ will certainly be ill conditioned if any of the eigenvalues $\lambda_{k+1}, \dots, \lambda_n$ are close to any of $\lambda_1, \dots, \lambda_k$. A necessary (but not sufficient) condition for well conditioning is that $\lambda_1, \dots, \lambda_k$ be well separated from $\lambda_{k+1}, \dots, \lambda_n$. A related practical fact is that if two eigenvalues are very close together, it may not be possible to swap them stably by LAPACK's TREXC.
34. (Performance) A 3.0 GHz Pentium 4 machine with 1 GB main memory and 1 MB cache computed the complete eigensystem of a random 1000×1000 real matrix using MATLAB in 56 seconds. This included balancing, reduction to upper Hessenberg form, triangularization by the QR algorithm, and back solving for the eigenvectors. All computed eigenpairs $(\lambda, \mathbf{v})$ satisfied $\|A\mathbf{v} - \lambda\mathbf{v}\|_1 < 10^{-15} \|A\|_1 \|\mathbf{v}\|_1$.
35. (Generalized eigenvalue problem) The steps for solving the dense, unsymmetric, generalized eigenvalue problem $A\mathbf{v} = \lambda B\mathbf{v}$ are analogous to those for solving the standard problem. First (optionally) the pair (A, B) is balanced (by routine GGBAL in LAPACK). Then it is transformed by a strictly unitary equivalence to a condensed form in which A is upper Hessenberg and B is upper triangular. Then the QZ algorithm completes the reduction to triangular form. Details are given in [GV96, § 7.7] and [Wat02, § 6.7]. In LAPACK, the codes GGHHD and HGEQZ reduce the pair to Hessenberg-triangular form and perform the QZ iterations, respectively.
36. Once A has been reduced to triangular form, the eigenvalues are $\lambda_j = a_{jj}/b_{jj}$, $j = 1, \dots, n$. The eigenvectors can be obtained by routines analogous to those used for the standard problem (LAPACK codes TGEVC and GGBAK), and condition numbers can be computed (LAPACK codes TGSNA and TGSEN).

Examples:

1. The matrix

$$A = \begin{bmatrix} -5.5849 \times 10^{-01} & -2.4075 \times 10^{+07} & -6.1644 \times 10^{+14} & 6.6275 \times 10^{+00} \\ -7.1724 \times 10^{-09} & -2.1248 \times 10^{+00} & -3.6183 \times 10^{+06} & 2.6435 \times 10^{-06} \\ -4.1508 \times 10^{-16} & -2.1647 \times 10^{-07} & 1.6229 \times 10^{-01} & -7.6315 \times 10^{-14} \\ 4.3648 \times 10^{-03} & 1.2614 \times 10^{+06} & -1.1986 \times 10^{+13} & -6.2002 \times 10^{-01} \end{bmatrix}$$

was balanced by Algorithm 1 (Fact 3) to produce

$$B = \begin{bmatrix} -0.5585 & -0.3587 & -1.0950 & 0.1036 \\ -0.4813 & -2.1248 & -0.4313 & 2.7719 \\ -0.2337 & -1.8158 & 0.1623 & -0.6713 \\ 0.2793 & 1.2029 & -1.3627 & -0.6200 \end{bmatrix}.$$

2. The matrix B of Example 1 was reduced to upper Hessenberg form by Algorithm 2 (Fact 7) to yield

$$H = \begin{bmatrix} -0.5585 & 0.7579 & 0.0908 & -0.8694 \\ 0.6036 & -3.2560 & -0.0825 & -1.8020 \\ 0 & 0.9777 & 1.2826 & -0.8298 \\ 0 & 0 & -1.5266 & -0.6091 \end{bmatrix}.$$

3. Algorithm 4 (Fact 23) was applied to the matrix H of Example 2, with $k = 1$ and shift $\mu_1 = h_{44} = -0.6091$, to produce

$$\begin{bmatrix} -3.1238 & -0.5257 & 1.0335 & 1.6798 \\ -1.3769 & 0.3051 & -1.5283 & 0.1296 \\ 0 & -1.4041 & 0.3261 & -1.0462 \\ 0 & 0 & -0.0473 & -0.6484 \end{bmatrix}.$$

The process was repeated twice again (with $\mu_1 = h_{44}$) to yield

$$\begin{bmatrix} -3.1219 & 0.7193 & 1.2718 & -1.4630 \\ 0.8637 & 1.8018 & 0.0868 & -0.3916 \\ 0 & 0.6770 & -1.2385 & 1.1642 \\ 0 & 0 & -0.0036 & -0.5824 \end{bmatrix}$$

and

$$\begin{bmatrix} -3.0939 & -0.6040 & 1.3771 & 1.2656 \\ -0.8305 & 1.8532 & -0.3517 & 0.5050 \\ 0 & 0.2000 & -1.3114 & -1.3478 \\ 0 & 0 & 0.00003 & -0.5888 \end{bmatrix}.$$

The (4,4) entry is an eigenvalue of A correct to four decimal places.

This matrix happens to have a real eigenvalue. If it had not, Algorithm 4 could have been used with $k = 2$ to extract the complex eigenvalues in pairs.

4. For an example of an ill-conditioned eigenvalue (Fact 31) consider a matrix

$$A = \begin{bmatrix} 1 & t \\ 0 & 1 + \epsilon \end{bmatrix},$$

where t is large or ϵ is small or both. Since A is upper triangular, its eigenvalues are 1 and $1 + \epsilon$.

Eigenvectors of A and A^T associated with the eigenvalue 1 are

$$\mathbf{x} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \text{and} \quad \mathbf{y} = \begin{bmatrix} 1 \\ -t/\epsilon \end{bmatrix},$$

respectively. Since $\|\mathbf{x}\|_2 = 1$, $\|\mathbf{y}\|_2 = \sqrt{1 + t^2/\epsilon^2}$, and $|\mathbf{y}^T \mathbf{x}| = 1$, the condition number of eigenvalue $\lambda = 1$ is $\kappa = \sqrt{1 + t^2/\epsilon^2} \approx t/\epsilon$. Thus if, for example, $t = 10^7$ and $\epsilon = 10^{-7}$, we have $\kappa \approx 10^{14}$.

5. This example illustrates Fact 32 on the ill conditioning of eigenvectors associated with a tight cluster of eigenvalues. Given a positive number ϵ that is as small as you please, the matrices

$$A_1 = \begin{bmatrix} 2 + \epsilon & 0 & 0 \\ 0 & 2 - \epsilon & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

and

$$A_2 = \begin{bmatrix} 2 & \epsilon & 0 \\ \epsilon & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

both have eigenvalues 1, $2 + \epsilon$, and $2 - \epsilon$, and they are very close together: $\|A_1 - A_2\|_2 = \sqrt{2}\epsilon$. However, unit eigenvectors associated with clustered eigenvalues $2 + \epsilon$ and $2 - \epsilon$ for A_1 are

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad \text{and} \quad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix},$$

while unit eigenvectors for A_2 are

$$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \quad \text{and} \quad \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}.$$

Thus, the tiny perturbation of order ϵ from A_1 to A_2 changes the eigenvectors completely; the eigenvectors are ill conditioned. In contrast the two-dimensional invariant subspace associated with the cluster $2 + \epsilon, 2 - \epsilon$ is $\text{Span}(\mathbf{e}_1, \mathbf{e}_2)$ for both A_1 and A_2 , and it is well conditioned.

43.3 Sparse Matrix Techniques

If the matrix A is large and sparse and just a few eigenvalues are needed, sparse matrix techniques are appropriate. Some examples of common tasks are: (1) find the few eigenvalues of largest modulus, (2) find the few eigenvalues with largest real part, and (3) find the few eigenvalues nearest some target value τ . The corresponding eigenvectors might also be wanted. These tasks are normally accomplished by computing the low-dimensional invariant subspace associated with the desired eigenvalues. Then the information about the eigenvalues and eigenvectors is extracted from the invariant subspace.

The most widely used method for the sparse unsymmetric eigenvalue problem is the implicitly restarted Arnoldi method, as implemented in ARPACK [LSY98], which is discussed in Chapter 76. A promising variant is the Krylov–Schur algorithm of Stewart [Ste01]. MATLAB's sparse eigenvalue command “eigs” calls ARPACK.

Definitions:

Given a subspace $\mathcal{S}$ of $\mathbb{C}^n$, a vector $\mathbf{v} \in \mathcal{S}$ is called a **Ritz vector** of A from $\mathcal{S}$ if there is a $\theta \in \mathbb{C}$ such that $A\mathbf{v} - \theta\mathbf{v} \perp \mathcal{S}$. The scalar θ is the **Ritz value** associated with $\mathcal{S}$. The pair $(\theta, \mathbf{v})$ is a **Ritz pair**.

Facts:

1. [Wat02, § 6.1] Let $\mathbf{v}_1, \dots, \mathbf{v}_m$ be a basis for a subspace $\mathcal{S}$ of $\mathbb{C}^n$, and let $V = [\mathbf{v}_1 \cdots \mathbf{v}_m]$. Then $\mathcal{S}$ is invariant under A if and only if there is a $B \in \mathbb{C}^{m \times m}$ such that $AV = VB$.
2. [Wat02, § 6.1] If $AV = VB$, then the eigenvalues of B are eigenvalues of A . If $\mathbf{x}$ is an eigenvector of B associated with eigenvalue μ , then $V\mathbf{x}$ is an eigenvector of A associated with μ .
3. [Wat02, § 6.4] Let $\mathbf{v}_1, \dots, \mathbf{v}_m$ be an orthonormal basis of $\mathcal{S}$, $V = [\mathbf{v}_1 \cdots \mathbf{v}_m]$, and $B = V^*AV$. Then the Ritz values of A associated with $\mathcal{S}$ are exactly the eigenvalues of B . If $(\theta, \mathbf{x})$ is an eigenpair of B , then $(\theta, V\mathbf{x})$ is a Ritz pair of A , and conversely.
4. If A is very large and sparse, it is essential to store A in a sparse data structure, in which only the nonzero entries of A are stored. One simple structure stores two integers n and nnz , which represent the dimension of the matrix and the number of nonzeros in the matrix, respectively. The matrix entries are stored in an array ent of length nnz , and the row and column indices are stored in two integer arrays of length nnz called row and col , respectively. For example, if the nonzero entry a_{ij} is stored in $ent(m)$, then this is indicated by setting $row(m) = i$ and $col(m) = j$. The space needed to store a matrix in this data structure is proportional to nnz .
5. Many operations that are routinely applied to dense matrices are impossible if the matrix is stored sparsely. Similarity transformations are out of the question because they quickly turn the zeros to nonzeros, transforming the sparse matrix to a full matrix.
6. One operation that is always possible is to multiply the matrix by a vector. This requires one pass through the data structure, and the work is proportional to nnz .

Algorithm 5. Sparse Matrix-Vector Multiply. Multiply A by $\mathbf{x}$ and store the result in $\mathbf{y}$.

$\mathbf{y} \leftarrow \mathbf{0}$

for $m = 1 : nnz$

$[\mathbf{y}(row(m)) \leftarrow \mathbf{y}(row(m)) + ent(m) * \mathbf{x}(col(m))]$

end

7. Because the matrix-vector multiply is so easy, many sparse matrix methods access the matrix A in only this way. At each step, A is multiplied by one or several vectors, and this is the only way A is used.
8. The following standard methodology is widely used. A starting vector $\mathbf{v}_1$ is chosen, and the algorithm adds one vector per step, so that after $j - 1$ steps it has produced j orthonormal vectors $\mathbf{v}_1, \dots, \mathbf{v}_j$. Let $V_j = [\mathbf{v}_1, \dots, \mathbf{v}_j] \in \mathbb{C}^{n \times j}$ and let $\mathcal{S}_j = \text{Span}(V_j) = \text{Span}(\mathbf{v}_1, \dots, \mathbf{v}_j)$. The j th step uses information from $\mathcal{S}_j$ to produce $\mathbf{v}_{j+1}$. The Ritz values of A associated with $\mathcal{S}_j$ are the eigenvalues of the $j \times j$ matrix $B_j = V_j^*AV_j$. The Ritz pair $(\theta, \mathbf{w})$ for which θ has the largest modulus is an estimate of the largest eigenvalue of A , and $\mathbf{x} = V_j\mathbf{w}$ is an estimate of the associated eigenvector. The residual $\mathbf{r}_j = A\mathbf{x} - \theta\mathbf{x}$ gives an indication of the quality of the approximate eigenpair.
9. Several methods use the residual $\mathbf{r}_j$ to help decide on the next basis vector $\mathbf{v}_{j+1}$. These methods typically use $\mathbf{r}_j$ to determine another vector $\mathbf{s}_j$, which is then orthonormalized against $\mathbf{v}_1, \dots, \mathbf{v}_j$ to produce $\mathbf{v}_{j+1}$. The choice $\mathbf{s}_j = \mathbf{r}_j$ leads to a method that is equivalent to the Arnoldi process. However, Arnoldi's process should not be implemented this way in practice. (See Chapter 44.) The choice $\mathbf{s}_j = (D - \theta I)^{-1}\mathbf{r}_j$, where D is the diagonal part of A , gives Davidson's method. The Jacobi–Davidson methods have more elaborate ways of choosing $\mathbf{s}_j$. (See [BDD00, § 7.12] for details.)
10. Periodic purging is employed to keep the dimension of the active subspace from becoming too large. Given m vectors, the purging process keeps the most promising k -dimensional subspace of

$\mathcal{S}_m = \text{Span}(V_m)$ and discards the rest. Again, let $B_m = V_m^* A V_m$, and let $B_m = U_m T_m U_m^*$ be a unitary similarity transformation to upper triangular form. The Ritz values lie on the main diagonal of T_m and can be placed in any order. Place the k most promising Ritz values at the top. Let $\tilde{V}_m = V_m U_m$, and let $\tilde{V}_k$ denote the $n \times k$ submatrix of $\tilde{V}_m$ consisting of the first k columns. The columns of $\tilde{V}_k$ are the vectors that are kept.

11. After each purge, the algorithm can be continued from step k . Once the basis has been expanded back to m vectors, another purge can be carried out. After a number of cycles of expansion and purging, the invariant subspace associated with the desired eigenvalues will have been found.
12. When purging is carried out in connection with the Arnoldi process, it is called an implicit restart, and there are some extra details. (See Chapter 44 and [Ste01]).
13. The implicitly restarted Arnoldi process is well suited for computing the eigenvalues on the periphery of the spectrum of A . Thus, it is good for computing the eigenvalues of maximum modulus or those of maximum or minimum real part.
14. For computing interior eigenvalues, the shift-and-invert strategy is often helpful. Suppose the eigenvalues nearest some target value τ are sought. The matrix $(A - \tau I)^{-1}$ has the same eigenvectors as A , but the eigenvalues are different. If $\lambda_1, \dots, \lambda_n$ are the eigenvalues of A , then $(\lambda_1 - \tau)^{-1}, \dots, (\lambda_n - \tau)^{-1}$ are the eigenvalues of $(A - \tau I)^{-1}$. The eigenvalues of $(A - \tau I)^{-1}$ of largest modulus correspond to the eigenvalues of A closest to τ . These can be computed by applying the implicitly restarted Arnoldi process to $(A - \tau I)^{-1}$. This is feasible whenever operations of the type $\mathbf{w} \leftarrow (A - \tau I)^{-1} \mathbf{x}$ can be performed efficiently. If a sparse decomposition $A - \tau I = P L U$ can be computed, as described in Chapter 41, then that decomposition can be used to perform the operation $\mathbf{w} \leftarrow (A - \tau I)^{-1} \mathbf{x}$ by back solves. If the $L U$ factors take up too much space to fit into memory, this method cannot be used.
15. Another option for solving $(A - \tau I) \mathbf{w} = \mathbf{x}$ is to use an iterative method, as described in Chapter 41. However, this is very computationally intensive, as the systems must be solved to high accuracy if the eigenvalues are to be computed accurately.
16. The shift-and-invert strategy can also be applied to the generalized eigenvalue problem $A \mathbf{v} = \lambda B \mathbf{v}$. The implicitly restarted Arnoldi process is applied to the operator $(A - \tau B)^{-1} B$ to find eigenvalues near τ .
17. If the matrix is too large for the shift-and-invert strategy, Jacobi–Davidson methods can be considered. These also require the iterative solution of linear systems. In this family of methods, inaccurate solution of the linear systems may slow convergence of the algorithm, but it will not cause the eigenvalues to be computed inaccurately.
18. Arnoldi-based and Jacobi–Davidson algorithms are described in [BDD00]. A brief overview is given in [Wat02, § 6.4]. Balancing of sparse matrices is discussed in [BDD00, § 7.2].

References

- [ABB99] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen. *LAPACK Users' Guide*, 3rd ed., SIAM, Philadelphia, 1999. www.netlib.org/lapack/lug/.
- [BD93] Z. Bai and J. W. Demmel. On swapping diagonal blocks in real Schur form. *Lin. Alg. Appl.* 186:73–95, 1993.
- [BDD00] Z. Bai, J. Demmel, J. Dongarra, A. Ruhe, and H. van der Vorst. *Templates for the Solution of Algebraic Eigenvalue Problems, a Practical Guide*. SIAM, Philadelphia, 2000.
- [BBM02a] K. Braman, R. Byers, and R. Mathias. The multi-shift QR algorithm part I: maintaining well-focused shifts and level 3 performance. *SIAM J. Matrix Anal. Appl.* 23:929–947, 2002.
- [BBM02b] K. Braman, R. Byers, and R. Mathias. The multi-shift QR algorithm part II: aggressive early deflation. *SIAM J. Matrix Anal. Appl.* 23:948–973, 2002.
- [Dem97] J.W. Demmel. *Applied Numerical Linear Algebra*. SIAM, Philadelphia, 1997.
- [GV96] G.H. Golub and C.F. Van Loan. *Matrix Computations*, 3rd ed., The Johns Hopkins University Press, Baltimore, MD, 1996.

- [Kre05] D. Kressner. *Numerical Methods for General and Structured Eigenproblems*. Springer, New York, 2005.
- [LSY98] R.B. Lehoucq, D.C. Sorensen, and C. Yang. *ARPACK Users' Guide*. SIAM, Philadelphia, 1998.
- [Osb60] E.E. Osborne. On pre-conditioning of matrices. *J. Assoc. Comput. Mach.* 7:338–345 1960.
- [PR69] B.N. Parlett and C. Reinsch. Balancing a matrix for calculation of eigenvalues and eigenvectors. *Numer. Math.* 13:293–304, 1969. Also published as contribution II/11 in [WR71].
- [Ste01] G.W. Stewart. A Krylov–Schur algorithm for large eigenproblems. *SIAM J. Matrix Anal. Appl.*, 23:601–614, 2001.
- [Wat02] D.S. Watkins. *Fundamentals of Matrix Computations*, 2nd ed., John Wiley & Sons, New York, 2002.
- [WE91] D.S. Watkins and L. Elsner. Convergence of algorithms of decomposition type for the eigenvalue problem. *Lin. Alg. Appl.* 143:19–47, 1991.
- [WR71] J.H. Wilkinson and C. Reinsch. *Handbook for Automatic Computation, Vol. II, Linear Algebra*, Springer-Verlag, New York, 1971.

The Implicitly Restarted Arnoldi Method

44.1	Krylov Subspace Projection.....	44-1
44.2	The Arnoldi Factorization	44-2
44.3	Restarting the Arnoldi Process	44-4
44.4	Polynomial Restarting	44-5
44.5	Implicit Restarting	44-6
44.6	Convergence of IRAM	44-9
44.7	Convergence in Gap: Distance to a Subspace	44-9
44.8	The Generalized Eigenproblem	44-11
44.9	Krylov Methods with Spectral Transformations	44-11
	References	44-12

D. C. Sorensen

Rice University

The implicitly restarted Arnoldi method (IRAM) [Sor92] is a variant of Arnoldi's method for computing a selected subset of eigenvalues and corresponding eigenvectors for large matrices. Implicit restarting is a synthesis of the implicitly shifted QR iteration and the Arnoldi process that effectively limits the dimension of the Krylov subspace required to obtain good approximations to desired eigenvalues. The space is repeatedly expanded and contracted with each new Krylov subspace generated by an updated starting vector obtained by implicit application of a matrix polynomial to the old starting vector. This process is designed to filter out undesirable components in the starting vector in a way that enables convergence to the desired invariant subspace. This method has been implemented and is freely available as ARPACK. The MATLAB[®] function `eigs` is based upon ARPACK. Use of this software is described in Chapter 76.

In this article, all matrices, vectors, and scalars are complex and the algorithms are phrased in terms of complex arithmetic. However, when the matrix (or matrix pair) happens to be real then the computations may be organized so that only real arithmetic is required. Multiplication of a vector $\mathbf{x}$ by a scalar λ is denoted by $\mathbf{x}\lambda$ so that the eigenvector–eigenvalue relation is $A\mathbf{x} = \mathbf{x}\lambda$. This convention provides for direct generalizations to the more general invariant subspace relations $AX = XH$, where X is an $n \times k$ matrix and H is a $k \times k$ matrix with $k < n$. More detailed discussion of all facts and definitions may be found in the overview article [Sor02].

44.1 Krylov Subspace Projection

The classic power method is the simplest way to compute the dominant eigenvalue and corresponding eigenvector of a large matrix. Krylov subspace projection provides a way to extract additional eigen-information from the power method iteration by considering all possible linear combinations of the sequence of vectors produced by the power method.